

```
// ===== backend/database.js =====

const mysql = require('mysql2/promise');
require('dotenv').config();

class Database {
  constructor() {
    this.pool = null;
    this.isInitializing = false;
    this.initPromise = null;
    this.retryTimer = null;
    this.init();
  }

  // 连接失败时进行指数退避重试，待 MySQL 恢复后自动重连（不再让进程崩溃）
  scheduleRetry() {
    if (this.retryTimer) return;
    const delay = 5000;
    console.log(`[DB] ${delay / 1000}s 后尝试重新连接数据库...`);
    this.retryTimer = setTimeout(() => {
      this.retryTimer = null;
      this.init().catch(() => {});
    }, delay);
  }

  async init() {
    // 如果已经在初始化，返回相同的Promise
    if (this.isInitializing) {
      return this.initPromise;
    }

    // 如果已经初始化成功，直接返回
    if (this.pool) {
      return Promise.resolve();
    }

    this.isInitializing = true;

    this.initPromise = (async () => {
      try {
        console.log('开始初始化数据库连接池...');
        console.log(`数据库配置: ${process.env.DB_HOST}:${process.env.DB_PORT}/${process.env.DB_NAME}`);

        this.pool = mysql.createPool({
          host: process.env.DB_HOST,
          port: parseInt(process.env.DB_PORT) || 3306,
          user: process.env.DB_USER,
          password: process.env.DB_PASSWORD,
          database: process.env.DB_NAME,
          connectionLimit: parseInt(process.env.DB_CONNECTION_LIMIT) || 30,
        });
      } catch (error) {
        console.error('数据库连接池初始化失败:', error);
        this.scheduleRetry();
      }
    })();
  }
}
```

```

    maxIdle: parseInt(process.env.DB_MAX_IDLE) || 15,
    connectTimeout: 10000,
    idleTimeout: 60000,
    charset: 'utf8mb4',
    timezone: '+08:00',
    dateStrings: true,
    multipleStatements: false,
    waitForConnections: true,
    queueLimit: 0,
    namedPlaceholders: false
  });

  // 捕获连接池级错误，避免未监听的 'error' 事件触发未捕获异常导致进程退出
  this.pool.on('error', (err) => {
    console.error('[DB] 连接池错误（已捕获，不崩溃进程）:', err.code
    || err.message);
    // 仅对连接断开类错误尝试自动重连；协议/语法错误不重试
    if (['PROTOCOL_CONNECTION_LOST', 'ECONNRESET', 'ETIMEDOUT', 'ENOTFOUND'].includes(err.code)) {
      this.scheduleRetry();
    }
  });

  // 测试连接
  const connection = await this.pool.getConnection();
  await connection.ping();

  // 自动创建必要的表（如果不存在）
  await this.ensureTables(connection);

  connection.release();

  console.log('数据库连接池初始化成功');
  this.isInitializing = false;
} catch (error) {
  console.error('数据库连接池初始化失败（将自动重试）:', error.message
);
  this.pool = null;
  this.isInitializing = false;
  // 吞掉异常，避免未捕获的 Promise 拒绝导致进程退出；改用定时重试
  this.scheduleRetry();
  throw error;
}
})();

// 防止构造函数里首次 init 的拒绝变成未捕获异常而崩溃进程
this.initPromise.catch(() => {});

return this.initPromise;
}

```

```

async getConnection() {
  // 等待初始化完成
  if (this.isInitializing) {
    await this.initPromise;
  }

  if (!this.pool) {
    throw new Error('数据库连接池未初始化，请检查数据库配置');
  }

  try {
    const connection = await this.pool.getConnection();

    // 统一数据库会话时区为东八区，避免 NOW()/CURRENT_TIMESTAMP 与业务时间不一致
    if (!connection.__timeZoneInitialized) {
      await connection.query('SET time_zone = '+08:00');
      connection.__timeZoneInitialized = true;
    }

    return connection;
  } catch (error) {
    console.error('获取数据库连接失败:', error);
    throw new Error('无法获取数据库连接: ' + error.message);
  }
}

async query(sql, params = []) {
  const connection = await this.getConnection();
  try {
    console.log('数据库查询 - SQL:', sql.replace(/\s+/g, ' ').trim());
    console.log('数据库查询 - 参数:', params);
    console.log('数据库查询 - 参数类型:', params.map(p => typeof p));

    const [rows] = await connection.query(sql, params);
    return rows;
  } catch (error) {
    console.error('数据库查询错误:', error);
    console.error('SQL:', sql);
    console.error('参数:', params);
    console.error('错误代码:', error.code);
    console.error('错误号:', error.errno);
    console.error('SQL状态:', error.sqlState);
    console.error('SQL消息:', error.sqlMessage);
    throw error;
  } finally {
    connection.release();
  }
}

```

```

}

async transaction(callback) {
  const connection = await this.getConnection();
  try {
    await connection.beginTransaction();
    const result = await callback(connection);
    await connection.commit();
    return result;
  } catch (error) {
    await connection.rollback();
    throw error;
  } finally {
    connection.release();
  }
}

// 自动建表（幂等，不存在才创建）
async ensureTables(connection) {
  const tables = [
    // 聊天会话表
    `CREATE TABLE IF NOT EXISTS chat_conversations (
      id VARCHAR(64) PRIMARY KEY,
      user_id INT NOT NULL,
      user_openid VARCHAR(128) DEFAULT '',
      status ENUM('active', 'transferred', 'completed') DEFAULT 'active',
      summary VARCHAR(512) DEFAULT '',
      context JSON,
      end_reason VARCHAR(64) DEFAULT '',
      last_activity DATETIME DEFAULT CURRENT_TIMESTAMP,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      INDEX idx_user_id (user_id),
      INDEX idx_status (status),
      INDEX idx_last_activity (last_activity)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci`,
    // 聊天消息表
    `CREATE TABLE IF NOT EXISTS chat_messages (
      id VARCHAR(64) PRIMARY KEY,
      conversation_id VARCHAR(64) NOT NULL,
      sender_type ENUM('user', 'ai', 'human', 'system') NOT NULL,
      content TEXT NOT NULL,
      entities JSON,
      intent VARCHAR(128) DEFAULT '',
      confidence DECIMAL(5,4) DEFAULT 0,
      suggested_actions JSON,
      reply_to_id VARCHAR(64) DEFAULT '',
      is_read TINYINT(1) DEFAULT 0,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      INDEX idx_conversation_id (conversation_id),
      INDEX idx_sender_type (sender_type),

```

```

    INDEX idx_created_at (created_at),
    FOREIGN KEY (conversation_id) REFERENCES chat_conversations(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci`,
// 人工客服队列表
`CREATE TABLE IF NOT EXISTS human_queue (
    id INT PRIMARY KEY AUTO_INCREMENT,
    conversation_id VARCHAR(64) NOT NULL,
    user_id INT NOT NULL,
    status ENUM('waiting', 'connected', 'completed', 'cancelled') DEFAULT 'waiting',
    priority VARCHAR(20) DEFAULT 'normal',
    reason VARCHAR(64) DEFAULT '',
    queue_position INT DEFAULT 0,
    estimated_wait_time INT DEFAULT 0,
    assigned_admin_id VARCHAR(64) DEFAULT '',
    assigned_admin_name VARCHAR(128) DEFAULT '',
    connected_at DATETIME NULL,
    completed_at DATETIME NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE KEY uniq_conversation (conversation_id),
    INDEX idx_status (status),
    INDEX idx_assigned_admin_id (assigned_admin_id),
    FOREIGN KEY (conversation_id) REFERENCES chat_conversations(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci`
];

for (const sql of tables) {
    try {
        await connection.query(sql);
    } catch (err) {
        console.error('建表警告 (已忽略) :', err.message);
    }
}

await this.ensureChatServiceSchema(connection);
await this.ensureAfterSalesSchema(connection);
await this.ensureFeedbackSchema(connection);
await this.ensureAdminCreatedSchema(connection);

// 同步设备类型表, 使其与前端 deviceData.js 的 deviceTypes 保持一致
// (扫描识别/添加设备时, 前端按 deviceData.js 的 id 传 device_type_id
,
// 若数据库缺少对应类型会导致添加设备 400 "设备类型不存在")
await this.ensureDeviceTypes(connection);
}

/**
 * 将 device_types 表与前端 utils/deviceData.js 的 deviceTypes
对齐 (幂等)。
 * - 表不存在则创建

```

```

* - 已存在的 id 更新名称/图标, 不存在的 id 插入
* - 保留 id=0 (自定义设备)
*/
async ensureDeviceTypes(connection) {
  try {
    await connection.query(`
      CREATE TABLE IF NOT EXISTS device_types (
        id INT PRIMARY KEY,
        name VARCHAR(64) NOT NULL,
        icon VARCHAR(16) DEFAULT '',
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    `);

    // 确保 id=0 自定义设备存在
    await connection.query(
      `INSERT IGNORE INTO device_types (id, name, icon)
      VALUES (0, '自定义设备', '□□')`
    );

    const SHOP = require('../utils/deviceData.js');
    const types = SHOP.deviceTypes || [];
    for (const t of types) {
      await connection.query(
        `INSERT INTO device_types (id, name, icon) VALUES (?, ?, ?)
        ON DUPLICATE KEY UPDATE name = VALUES(name), icon = VALUES(icon)`,
        [t.id, t.name, t.icon || '']
      );
    }
    console.log(`[DB] device_types 已与 deviceData.js 同步, 共
    ${types.length} 个类型`);
  } catch (err) {
    console.error(`[DB] 同步 device_types 失败 (已忽略) :`, err.m
    essage);
  }
}

/**
 * 管理员代客下单相关字段 (幂等)
 * - orders 增加 is_admin_created / admin_created_by / adm
in_created_at
 * - status 为 VARCHAR, admin_created 直接可作为状态值使用, 无需改表结构
 */
async ensureAdminCreatedSchema(connection) {
  const alter = [];
  if (!await this.columnExists(connection, 'orders', 'is_admin_created')) {
    alter.push(`ALTER TABLE orders ADD COLUMN is_admin_c
    reated TINYINT(1) NOT NULL DEFAULT 0 COMMENT '是否管理员代客下单 (跳过
    报价环节, 直接设定金额) '`);
  }
}

```

```

    if (!await this.columnExists(connection, 'orders', 'admin_created_by')) {
      alter.push(`ALTER TABLE orders ADD COLUMN admin_created_by INT NULL COMMENT '代客下单的管理员ID'`);
    }
    if (!await this.columnExists(connection, 'orders', 'admin_created_at')) {
      alter.push(`ALTER TABLE orders ADD COLUMN admin_created_at DATETIME NULL COMMENT '代客下单时间'`);
    }

    for (const sql of alter) {
      try {
        await connection.query(sql);
      } catch (err) {
        console.error(`[DB] 代客下单表结构修正警告 (已忽略) :`, err.message);
      }
    }
    if (alter.length > 0) {
      console.log(`[DB] 已为 orders 表增加管理员代客下单字段`);
    }
  }

  async columnExists(connection, tableName, columnName) {
    const [rows] = await connection.query(
      `SELECT COUNT(*) AS count
      FROM INFORMATION_SCHEMA.COLUMNS
      WHERE TABLE_SCHEMA = DATABASE()
      AND TABLE_NAME = ?
      AND COLUMN_NAME = ?`,
      [tableName, columnName]
    );
    return Number(rows[0]?.count || 0) > 0;
  }

  async indexExists(connection, tableName, indexName) {
    const [rows] = await connection.query(
      `SELECT COUNT(*) AS count
      FROM INFORMATION_SCHEMA.STATISTICS
      WHERE TABLE_SCHEMA = DATABASE()
      AND TABLE_NAME = ?
      AND INDEX_NAME = ?`,
      [tableName, indexName]
    );
    return Number(rows[0]?.count || 0) > 0;
  }

  async ensureChatServiceSchema(connection) {
    const alterStatements = [];

    if (await this.columnExists(connection, 'human_queue', 'reason') === false) {

```

```

    alterStatements.push(`ALTER TABLE human_queue ADD COLUMN reason VARCHAR(64) DEFAULT '' AFTER priority`
);
}

if (await this.columnExists(connection, 'human_queue', 'assigned_admin_id') === false) {
    alterStatements.push(`ALTER TABLE human_queue ADD COLUMN assigned_admin_id VARCHAR(64) DEFAULT '' AFTER estimated_wait_time`);
}

if (await this.columnExists(connection, 'human_queue', 'assigned_admin_name') === false) {
    alterStatements.push(`ALTER TABLE human_queue ADD COLUMN assigned_admin_name VARCHAR(128) DEFAULT '' AFTER assigned_admin_id`);
}

if (await this.columnExists(connection, 'human_queue', 'connected_at') === false) {
    alterStatements.push(`ALTER TABLE human_queue ADD COLUMN connected_at DATETIME NULL AFTER assigned_admin_name`);
}

if (await this.columnExists(connection, 'human_queue', 'completed_at') === false) {
    alterStatements.push(`ALTER TABLE human_queue ADD COLUMN completed_at DATETIME NULL AFTER connected_at`);
}

alterStatements.push(
    `ALTER TABLE human_queue MODIFY COLUMN status ENUM('waiting','assigned','connected','completed','cancelled') DEFAULT 'waiting'`
);

if (await this.indexExists(connection, 'human_queue', 'uniq_conversation') === false) {
    alterStatements.push(`ALTER TABLE human_queue ADD UNIQUE INDEX uniq_conversation (conversation_id)`);
}

if (await this.indexExists(connection, 'human_queue', 'idx_assigned_admin_id') === false) {
    alterStatements.push(`ALTER TABLE human_queue ADD INDEX idx_assigned_admin_id (assigned_admin_id)`);
}

for (const sql of alterStatements) {
    try {
        await connection.query(sql);
    } catch (err) {
        console.error('客服表结构修正警告 (已忽略) :', err.message);
    }
}

/**
 * 售后/质保相关表结构 (幂等)
 * - orders 增加 device_id (关联用户设备)、质保字段、是否质保单/原单关联
 * - user_devices 增加默认质保月数

```



```

*/
async ensureAfterSalesSchema(connection) {
  const alter = [];
  if (!await this.columnExists(connection, 'orders', 'device_id')) {
    alter.push(`ALTER TABLE orders ADD COLUMN device_id
INT NULL COMMENT '关联用户设备ID(user_devices.id)' AFTER user_id
`);
  }
  if (!await this.columnExists(connection, 'orders', 'warranty_start_date')) {
    alter.push(`ALTER TABLE orders ADD COLUMN warranty_s
tart_date DATE NULL COMMENT '质保起始日'`);
  }
  if (!await this.columnExists(connection, 'orders', 'warranty_end_date')) {
    alter.push(`ALTER TABLE orders ADD COLUMN warranty_e
nd_date DATE NULL COMMENT '质保到期日'`);
  }
  if (!await this.columnExists(connection, 'orders', 'warranty_period_days')) {
    alter.push(`ALTER TABLE orders ADD COLUMN warranty_p
eriod_days INT NULL COMMENT '质保天数'`);
  }
  if (!await this.columnExists(connection, 'orders', 'warranty_type')) {
    alter.push(`ALTER TABLE orders ADD COLUMN warranty_t
ype VARCHAR(32) NULL COMMENT '质保类型: 整备/返修'`);
  }
  if (!await this.columnExists(connection, 'orders', 'is_warranty')) {
    alter.push(`ALTER TABLE orders ADD COLUMN is_warrant
y TINYINT(1) NOT NULL DEFAULT 0 COMMENT '是否质保/返修单'`);
  }
  if (!await this.columnExists(connection, 'orders', 'original_order_id')) {
    alter.push(`ALTER TABLE orders ADD COLUMN original_o
rder_id INT NULL COMMENT '原始订单ID(返修关联)'`);
  }
  if (!await this.columnExists(connection, 'user_devices', 'warranty_months')) {
    alter.push(`ALTER TABLE user_devices ADD COLUMN warr
anty_months INT NULL DEFAULT 3 COMMENT '默认质保月数'`);
  }
  // 设备用途标签: repair=仅用于维修, recycle=仅用于回收, both=维修/回收均可 (默认)
  // 同一台设备只存一条记录, 维修设备清单与回收设备清单都是它的视图, 天然同步一致
  if (!await this.columnExists(connection, 'user_devices', 'device_purpose')) {
    alter.push(`ALTER TABLE user_devices ADD COLUMN devi
ce_purpose VARCHAR(16) NOT NULL DEFAULT 'both' COMMENT '设备
用途: repair维修/recycle回收/both均可'`);
  }

  for (const sql of alter) {
    try {
      await connection.query(sql);
    } catch (err) {
      console.error('[DB] 售后表结构修正警告 (已忽略) :', err.message)
    }
  }
;

```

```

    }
}

// 售后申请工单表（用户/管理员针对已完成、待评价订单的具体产品发起售后）
try {
  await connection.query(`
    CREATE TABLE IF NOT EXISTS after_sales_requests (
      id INT PRIMARY KEY AUTO_INCREMENT,
      order_id INT NOT NULL COMMENT '关联订单ID',
      user_id INT NOT NULL COMMENT '发起人ID(用户或管理员)',
      product_name VARCHAR(255) NOT NULL COMMENT '具体产品名称',
      product_model VARCHAR(255) DEFAULT '' COMMENT '产品型号',
      type VARCHAR(32) NOT NULL DEFAULT 'repair' COMMENT '售后类型: repair维修/replace换货/return退货/other其他',
      description TEXT COMMENT '问题描述',
      contact_phone VARCHAR(32) DEFAULT '' COMMENT '联系电话',
      images JSON COMMENT '图片凭证',
      status ENUM('pending','processing','resolved','rejected') NOT NULL DEFAULT 'pending' COMMENT '状态',
      admin_remark TEXT COMMENT '处理说明/解决方案',
      resolved_at DATETIME NULL,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
      INDEX idx_order_id (order_id),
      INDEX idx_status (status),
      INDEX idx_user_id (user_id)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  `);
  console.log('[DB] after_sales_requests 表已就绪');
} catch (err) {
  console.error('[DB] 创建 after_sales_requests 表警告（已忽略）');
};

/**
 * 意见反馈表（幂等）
 * - 用户提交意见/建议，管理员回复并标记处理状态
 */
async ensureFeedbackSchema(connection) {
  try {
    await connection.query(`
      CREATE TABLE IF NOT EXISTS feedback (
        id INT PRIMARY KEY AUTO_INCREMENT,
        user_id INT NOT NULL COMMENT '提交用户ID',
        type VARCHAR(32) NOT NULL DEFAULT 'suggestion' COMMENT '反馈类型: suggestion功能建议/complaint问题投诉/other其他',

```

```

        content TEXT NOT NULL COMMENT '反馈内容',
        contact VARCHAR(64) DEFAULT '' COMMENT '联系方式 (手机号/微信号)',
        status ENUM('pending','replied','closed') NOT NULL DEFAULT 'pending' COMMENT '状态: pending待处理/replied已回复/closed已关闭',
        admin_reply TEXT COMMENT '管理员回复内容',
        replied_by INT NULL COMMENT '回复管理员ID',
        replied_at DATETIME NULL COMMENT '回复时间',
        closed_by INT NULL COMMENT '关闭操作管理员ID',
        closed_at DATETIME NULL COMMENT '关闭时间',
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        INDEX idx_user_id (user_id),
        INDEX idx_status (status)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    ');
    console.log('[DB] feedback 表已就绪');
} catch (err) {
    console.error('[DB] 创建 feedback 表警告 (已忽略) :', err.message);
}

// 检查连接状态
async checkConnection() {
    if (!this.pool) {
        return { connected: false, error: '连接池未初始化' };
    }

    try {
        const connection = await this.pool.getConnection();
        await connection.ping();
        connection.release();
        return { connected: true };
    } catch (error) {
        return { connected: false, error: error.message };
    }
}

module.exports = new Database();

```

```

// ===== backend/server.js =====
require('express-async-errors');
const express = require('express');
const cors = require('cors');
const path = require('path');
const http = require('http');
const WebSocket = require('ws');

```

```

const fs = require('fs');
const dotenv = require('dotenv');
require('dotenv').config({ path: path.join(__dirname, './.env') });
require('dotenv').config({ path: path.join(__dirname, '.env'), override: true });
const socketHub = require('./services/socketHub');

const app = express();
const db = require('./database');
const server = http.createServer(app);

// 统一静态文件服务：uploads 目录必须与上传写入目录保持一致。
// 服务器 Docker 实际挂载：宿主机 小程序2.0/uploads → 容器 /app/uploads（见用户提供的 miniprogram-backend compose volumes）。
// progressRoutes 写盘路径为 routes/./uploads = 容器 /app/uploads
// 因此主静态目录必须也是 /app/uploads
// （server.js 位于 /app，故 __dirname/uploads = /app/uploads）。
const uploadsRoot = path.join(__dirname, 'uploads');
// 静态资源启用浏览器缓存（文件名含时间戳、内容不可变，maxAge 安全；显著减少图片重复下载）
const staticOptions = { maxAge: '7d', immutable: true };
app.use('/uploads', express.static(uploadsRoot, staticOptions));
// 冗余兜底：同时通过 API 前缀暴露，避免 nginx 仅反代 /mp-api 时根路径 /uploads 取不到文件
app.use('/api/uploads', express.static(uploadsRoot, staticOptions));
// 兼容兜底：旧部署（电子维修2.0，已删除）曾把宿主机 uploads 挂到容器根 /uploads，
// 若容器内仍有历史残留文件可被读取。express.static 在文件不存在时会 next() 到下一层，正常
// 目录优先，互不影响。
const legacyUploadsRoot = path.join(__dirname, './uploads');
app.use('/uploads', express.static(legacyUploadsRoot, staticOptions));
app.use('/api/uploads', express.static(legacyUploadsRoot, staticOptions));

// 中间件
app.use(cors());
app.use('/api/pay/notify', express.text({ type: '*' }));
app.use('/api/pay/refund/notify', express.text({ type: '*' }));
// strict:false 允许请求体为顶层原始值（如微信 DELETE 发送 body="null" 时不会被拒绝）
app.use(express.json({ limit: '10mb', strict: false }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));

// 数据库健康检查
app.get('/db-health', async (req, res) => {
  try {
    const result = await db.query('SELECT 1 as status');
    res.json({ status: 'OK', database: 'connected', timestamp: new Date().toISOString() });
  } catch (error) {
    console.error('数据库健康检查失败:', error);
    res.status(500).json({ status: 'ERROR', database: 'disconnected', error: error.message });
  }
});

```

// 路由

```
const userRoutes = require('./routes/userRoutes');
const addressRoutes = require('./routes/addressRoutes');
const unitRoutes = require('./routes/unitRoutes');
const orderRoutes = require('./routes/orderRoutes');
const chatRoutes = require('./routes/chatRoutes');
const chatPersistenceRoutes = require('./routes/chatPersistenceRoutes');
const productRoutes = require('./routes/productRoutes');
const knowledgeRoutes = require('./routes/knowledgeRoutes');
const uploadRoutes = require('./routes/uploadRoutes');
const locationRoutes = require('./routes/locationRoutes');
const adminRoutes = require('./routes/adminRoutes');
const adminServiceRoutes = require('./routes/adminServiceRoutes');
const superAdminRoutes = require('./routes/superAdminRoutes');
const repairRecordsRoutes = require('./routes/repairRecordsRoutes');
const deliveryRoutes = require('./routes/deliveryRoutes');
const progressRoutes = require('./routes/progressRoutes');
const progressApplyRoutes = require('./routes/progressApplyRoutes');
const paymentRoutes = require('./routes/paymentRoutes');
const diagnoseRoutes = require('./routes/diagnoseRoutes');
const userDevicesRoutes = require('./routes/userDevicesRoutes');
const recycleRoutes = require('./routes/recycleRoutes');
const scanRoutes = require('./routes/scanRoutes');
const afterSalesRoutes = require('./routes/afterSalesRoutes');
const feedbackRoutes = require('./routes/feedbackRoutes');
const { expireOldReviewOrders } = require('./utils/reviewExpire');
const { ensureIncomeTable, backfillIncome } = require('./services/incomeService');
```

```
app.use('/api/user', userRoutes);
app.use('/api/user-devices', userDevicesRoutes);
app.use('/api/addresses', addressRoutes);
app.use('/api/units', unitRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/chat', chatRoutes);
app.use('/api/chat-persistence', chatPersistenceRoutes);
app.use('/api/products', productRoutes);
app.use('/api/knowledge', knowledgeRoutes);
app.use('/api/upload', uploadRoutes);
app.use('/api/location', locationRoutes);
app.use('/api/admin', adminRoutes);
app.use('/api/admin/service', adminServiceRoutes);
app.use('/api/super-admin', superAdminRoutes);
app.use('/api/repair-records', repairRecordsRoutes);
app.use('/api/delivery', deliveryRoutes);
app.use('/api/progress', progressRoutes);
app.use('/api/progress-apply', progressApplyRoutes);
app.use('/api/pay', paymentRoutes);
app.use('/api/diagnose', diagnoseRoutes);
app.use('/api/recycle', recycleRoutes);
app.use('/api/scan', scanRoutes);
```

```

app.use('/api/after-sales', afterSalesRoutes);
app.use('/api/feedback', feedbackRoutes);

// 健康检查
app.get('/health', (req, res) => {
  res.json({ status: 'OK', timestamp: new Date().toISOString() });
});

// 错误处理中间件
app.use((err, req, res, next) => {
  console.error('Error:', err);
  // 透传错误的真实状态码 (如 body-parser 的 400) , 不要再硬编码 500
  const status = err.status || err.statusCode || 500;
  res.status(status).json({
    error: status === 500 ? 'Internal Server Error' : err.type || 'Request Error',
    message: process.env.NODE_ENV === 'development' ? err.message : undefined
  });
});

// 404处理
app.use('*', (req, res) => {
  res.status(404).json({ error: 'Not Found' });
});

const PORT = process.env.PORT || 3001;
const SERVER_HOST = process.env.SERVER_HOST || '0.0.0.0';
const wss = new WebSocket.Server({ server, path: '/ws/chat' });
socketHub.attach(wss);

function readEnvSnapshot() {
  const rootEnvPath = path.join(__dirname, '../.env');
  const backendEnvPath = path.join(__dirname, '.env');
  const parse = (filePath) => {
    try {
      if (!fs.existsSync(filePath)) return {};
      return dotenv.parse(fs.readFileSync(filePath, 'utf8'));
    } catch (error) {
      return {};
    }
  };

  return {
    ...process.env,
    ...parse(rootEnvPath),
    ...parse(backendEnvPath)
  };
}

server.listen(PORT, SERVER_HOST, () => {
  const envSnapshot = readEnvSnapshot();

```

```

console.log(`Server is running on http://${SERVER_HOST}:${PORT}`);
console.log('[VoiceConfig]', {
  provider: envSnapshot.VOICE_PROVIDER || 'qwen',
  hasDashscopeKey: !(envSnapshot.DASHSCOPE_API_KEY || envSnapshot.VOICE_API_KEY),
  workspaceId: envSnapshot.DASHSCOPE_WORKSPACE_ID || '',
  voiceApiBaseUrl: envSnapshot.VOICE_API_BASE_URL || '',
  voiceModel: envSnapshot.VOICE_MODEL || 'qwen3-asr-flash'
});

// 待评价订单自动过期：启动即执行一次，之后每 30 分钟执行一次
expireOldReviewOrders();
setInterval(expireOldReviewOrders, 30 * 60 * 1000);

// 交易收入表：建表 + 历史已完成订单回填
ensureIncomeTable()
  .then(() => backfillIncome())
  .then((count) => console.log(`[收入表] 初始化完成，本次回填 ${count}
|| 0) 条历史收入`))
  .catch((e) => console.error(`[收入表] 初始化失败:`, e));
});

// ===== backend/middleware/adminAuth.js =====
const express = require('express');
const router = express.Router();
const db = require('../database');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');

// 角色常量定义
const ROLES = {
  USER: 'user', // 普通用户
  ADMIN: 'admin', // 管理员
  SUPER_ADMIN: 'super_admin', // 超级管理员
  INTERNAL: 'internal', // 公司内部人员（发起维修/回收订单免付款，仅需申请）
  REPAIR: 'repair' // 维修人员（负责处理维修/回收工单）
};

// 权限等级映射（数值越大权限越高）
const ROLE_LEVELS = {
  [ROLES.USER]: 1,
  [ROLES.INTERNAL]: 1, // 内部人员视为普通用户等级（仅免付款下单，无管理权限）
  [ROLES.REPAIR]: 2, // 维修人员可处理工单，等同管理员业务等级
  [ROLES.ADMIN]: 2,
  [ROLES.SUPER_ADMIN]: 3
};

/**
 * 生成JWT Token
 */
const generateToken = (user) => {

```

```

return jwt.sign(
  {
    userId: user.id,
    openid: user.openid,
    role: user.role || ROLES.USER,
    nickname: user.nickname
  },
  process.env.JWT_SECRET,
  { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }
);
};

/**
 * JWT验证中间件 - 通用认证
 */
const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: '需要提供访问令牌' });
  }

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ error: '无效或过期的令牌' });
    }
    req.user = user;
    next();
  });
};

/**
 * 角色验证中间件 - 检查用户是否具有指定角色
 */
const authorizeRole = (allowedRoles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: '未认证用户' });
    }

    const userRole = req.user.role || ROLES.USER;
    const userLevel = ROLE_LEVELS[userRole] || 0;

    // 检查用户角色是否在允许的角色列表中，或权限等级是否足够
    const hasAccess = allowedRoles.some(role => {
      const allowedLevel = ROLE_LEVELS[role] || 0;
      return userLevel >= allowedLevel;
    });
  };
};

```



```

    if (!hasAccess) {
      return res.status(403).json({
        error: '权限不足',
        required: allowedRoles,
        userRole: userRole
      });
    }
  }

  next();
};

/**
 * 仅限超级管理员访问
 */
const superAdminOnly = (req, res, next) => {
  if (!req.user || req.user.role !== ROLES.SUPER_ADMIN) {
    return res.status(403).json({ error: '需要超级管理员权限' });
  }
  next();
};

module.exports = {
  ROLES,
  authenticateToken,
  authorizeRole,
  superAdminOnly,
  generateToken
};

// ===== backend/middleware/auth.js =====
const jwt = require('jsonwebtoken');
const db = require('../database');

/**
 * 角色常量
 */
const ROLES = {
  USER: 'user',
  ADMIN: 'admin',
  SUPER_ADMIN: 'super_admin',
  INTERNAL: 'internal', // 公司内部人员（发起维修/回收订单免付款，仅需申请）
  REPAIR: 'repair' // 维修人员（负责处理维修/回收工单）
};

/**
 * JWT验证中间件
 * 验证Token是否有效，并将用户信息解析到 req.user
 */
const authenticateToken = async (req, res, next) => {

```

```

try {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
// 格式: Bearer TOKEN

  if (!token) {
    return res.status(401).json({
      success: false,
      error: '需要提供访问令牌'
    });
  }

  const decoded = jwt.verify(token, process.env.JWT_SECRET || '**** ... ****');

  // 从数据库获取最新用户信息
  const users = await db.query(
    'SELECT id, openid, nickname, real_name, phone, email, avatar_url, role, status FROM users WHERE id =
?',
    [decoded.userId]
  );

  if (users.length === 0) {
    return res.status(401).json({
      success: false,
      error: '用户不存在'
    });
  }

  const user = users[0];

  if (user.status !== 1) {
    return res.status(403).json({
      success: false,
      error: '用户已被禁用'
    });
  }

  req.user = user;
  req.token = token;
  next();
} catch (error) {
  console.error('Token验证错误:', error);
  return res.status(401).json({
    success: false,
    error: '无效或过期的令牌'
  });
}
};

/**

```

```

* 生成JWT Token
*/
const generateToken = (user) => {
  const payload = {
    userId: user.id,
    openid: user.openid,
    nickname: user.nickname,
    role: user.role
  };

  const options = {
    expiresIn: process.env.JWT_EXPIRES_IN || '7d'
  };

  return jwt.sign(payload, process.env.JWT_SECRET || '**** ... ****', options);
};

/**
 * 管理员权限验证中间件
 * 只有管理员角色才能访问
 */
const requireAdmin = (req, res, next) => {
  if (!req.user) {
    return res.status(401).json({
      success: false,
      error: '需要先登录'
    });
  }

  if (req.user.role !== ROLES.ADMIN && req.user.role !== ROLES.SUPER_ADMIN) {
    return res.status(403).json({
      success: false,
      error: '需要管理员权限',
      currentRole: req.user.role
    });
  }

  next();
};

/**
 * 超级管理员权限验证中间件
 * 只有超级管理员角色才能访问
 */
const requireSuperAdmin = (req, res, next) => {
  if (!req.user) {
    return res.status(401).json({
      success: false,
      error: '需要先登录'
    });
  }

```

```

}

if (req.user.role !== ROLES.SUPER_ADMIN) {
  return res.status(403).json({
    success: false,
    error: '需要超级管理员权限',
    currentRole: req.user.role
  });
}

next();
};

/**
 * 角色权限验证中间件
 * @param {string|Array} allowedRoles - 允许的角色或角色数组
 */
const authorizeRole = (allowedRoles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({
        success: false,
        error: '需要先登录'
      });
    }

    const roles = Array.isArray(allowedRoles) ? allowedRoles : [allowedRoles];

    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        error: '权限不足',
        currentRole: req.user.role,
        allowedRoles: roles
      });
    }

    next();
  };
};

module.exports = {
  ROLES,
  authenticateToken,
  generateToken,
  requireAdmin,
  requireSuperAdmin,
  authorizeRole
};

```

```
// ===== backend/middleware/imageCompress.js =====
```

```
const sharp = require('sharp');
const path = require('path');
const fs = require('fs');

/**
 * 压缩图片中间件
 * @param {Object} file - 上传的文件对象
 * @param {Object} options - 压缩选项
 * @returns {Promise<Object>} 压缩后的文件信息
 */
async function compressImage(file, options = {}) {
  const {
    maxWidth = 200,
    maxHeight = 200,
    quality = 80,
    format = 'jpeg',
    fit = 'cover'
  } = options;

  try {
    const inputPath = file.path;
    const filename = file.filename;
    const uploadDir = path.dirname(inputPath);

    // 生成压缩后的文件名
    const compressedFilename = `compressed-${filename.replace(/\.([^.]*)$/, '')}.${format}`;
    const compressedPath = path.join(uploadDir, compressedFilename);

    // 使用sharp进行压缩
    let transform = sharp(inputPath)
      .resize(maxWidth, maxHeight, {
        fit,
        position: 'center'
      });

    // 根据格式设置质量
    if (format === 'jpeg') {
      transform = transform.jpeg({ quality });
    } else if (format === 'png') {
      transform = transform.png({ quality });
    } else if (format === 'webp') {
      transform = transform.webp({ quality });
    }

    // 保存压缩后的图片
    await transform.toFile(compressedPath);

    // 获取文件大小
    const originalStats = fs.statSync(inputPath);
```

```

const compressedStats = fs.statSync(compressedPath);

console.log(`图片压缩完成:`, {
  original: `${(originalStats.size / 1024).toFixed(2)} KB`,
  compressed: `${(compressedStats.size / 1024).toFixed(2)} KB`,
  ratio: `${((1 - compressedStats.size / originalStats
.size) * 100).toFixed(2)}% 压缩率`
});

// 删除原始文件
fs.unlinkSync(inputPath);

return {
  ...file,
  filename: compressedFilename,
  path: compressedPath,
  size: compressedStats.size
};
} catch (error) {
  console.error('图片压缩失败:', error);
  throw error;
}
}

/**
 * Multer存储引擎, 包含自动压缩
 */
const createCompressedStorage = (uploadDir) => {
  return {
    destination: (req, file, cb) => {
      cb(null, uploadDir);
    },
    filename: (req, file, cb) => {
      const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
      cb(null, req.user?.userId + '-' + uniqueSuffix + path.extname(file.originalname));
    }
  };
};

module.exports = {
  compressImage,
  createCompressedStorage
};

// ===== backend/routes/addressRoutes.js =====
// backend/routes/addressRoutes.js
const express = require('express');
const router = express.Router();
const db = require('../database');
const { authenticateToken } = require('../middleware/auth');
```

```

// 获取地址列表
router.get('/', authenticateToken, async (req, res) => {
  try {
    console.log('获取地址列表 - userId:', req.user.id);
    const addresses = await db.query(
      'SELECT * FROM user_addresses WHERE user_id = ? ORDER BY is_default DESC, created_at DESC',
      [req.user.id]
    );

    console.log('查询到的地址数量:', addresses.length);
    res.json(addresses || []);
  } catch (error) {
    console.error('获取地址列表失败:', error);
    console.error('错误详情:', error.message);
    res.status(500).json({ error: '获取地址列表失败', message: error.message });
  }
});

// 创建地址
router.post('/', authenticateToken, async (req, res) => {
  try {
    const { contact_name, contact_phone, province, city, district, detail_address, postal_code, tags, is_default } = req.body;

    // 验证必填字段
    if (!contact_name || !contact_phone || !province || !city || !district || !detail_address) {
      return res.status(400).json({ error: '缺少必填字段' });
    }

    // 如果设置为默认地址，先取消其他地址的默认状态
    if (is_default) {
      await db.query(
        'UPDATE user_addresses SET is_default = 0 WHERE user_id = ?',
        [req.user.id]
      );
    }

    const result = await db.query(
      `INSERT INTO user_addresses (user_id, contact_name, contact_phone, province, city, district, detail_address, postal_code, tags, is_default)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`
    );

    [
      req.user.id,
      contact_name,
      contact_phone,
      province,
      city,
      district,

```

```

    detail_address,
    postal_code,
    JSON.stringify(tags || []),
    is_default || false
  ]
};

const newAddress = await db.query(
  'SELECT * FROM user_addresses WHERE id = ?',
  [result.insertId]
);

res.json(newAddress[0]);
} catch (error) {
  console.error('创建地址失败:', error);
  res.status(500).json({ error: '创建地址失败', message: error
.message });
}
});

// 更新地址
router.put('/:id', authenticateToken, async (req, res) => {
  try {
    const addressId = req.params.id;
    const { contact_name, contact_phone, province, city, district, detail_address, postal_code, tags, is_def
ault } = req.body;

    // 验证地址是否属于当前用户
    const existingAddress = await db.query(
      'SELECT * FROM user_addresses WHERE id = ? AND user_id = ?',
      [addressId, req.user.id]
    );

    if (existingAddress.length === 0) {
      return res.status(404).json({ error: '地址不存在' });
    }

    // 如果设置为默认地址，先取消其他地址的默认状态
    if (is_default) {
      await db.query(
        'UPDATE user_addresses SET is_default = 0 WHERE user_id = ?',
        [req.user.id]
      );
    }

    await db.query(
      `UPDATE user_addresses
      SET contact_name = ?, contact_phone = ?, province = ?, city = ?, district = ?, detail_address = ?, po
stal_code = ?, tags = ?, is_default = ?, updated_at = CURRENT_TIMESTAMP
      WHERE id = ?`,

```



```

[
  contact_name,
  contact_phone,
  province,
  city,
  district,
  detail_address,
  postal_code,
  JSON.stringify(tags || []),
  is_default || false,
  addressId
]
);

const updatedAddress = await db.query(
  'SELECT * FROM user_addresses WHERE id = ?',
  [addressId]
);

res.json(updatedAddress[0]);
} catch (error) {
  console.error('更新地址失败:', error);
  res.status(500).json({ error: '更新地址失败', message: error
.message });
}
});

// 删除地址
router.delete('/:id', authenticateToken, async (req, res) => {
  try {
    const addressId = req.params.id;

    // 验证地址是否属于当前用户
    const existingAddress = await db.query(
      'SELECT * FROM user_addresses WHERE id = ? AND user_id = ?',
      [addressId, req.user.id]
    );

    if (existingAddress.length === 0) {
      return res.status(404).json({ error: '地址不存在' });
    }

    await db.query(
      'DELETE FROM user_addresses WHERE id = ?',
      [addressId]
    );

    res.json({ success: true });
  } catch (error) {
    console.error('删除地址失败:', error);
  }
});

```

```

    res.status(500).json({ error: '删除地址失败', message: error
.message });
  }
});

// 设置默认地址
router.post('/:id/default', authenticateToken, async (req, res) => {
  try {
    const addressId = req.params.id;
    const userId = req.user.id;

    console.log('[设置默认地址] addressId:', addressId, 'userId:
', userId);

    // 验证地址是否属于当前用户
    const existingAddress = await db.query(
      'SELECT id FROM user_addresses WHERE id = ? AND user_id = ?',
      [addressId, userId]
    );

    if (existingAddress.length === 0) {
      return res.status(404).json({ success: false, error:
'地址不存在' });
    }

    // 使用事务保证原子性
    await db.transaction(async (connection) => {
      // 取消该用户所有地址的默认状态
      await connection.query(
        'UPDATE user_addresses SET is_default = 0 WHERE user_id = ?',
        [userId]
      );

      // 设置当前地址为默认
      await connection.query(
        'UPDATE user_addresses SET is_default = 1, updated_at = CURRENT_TIMESTAMP WHERE id = ?',
        [addressId]
      );
    });

    res.json({ success: true });
  } catch (error) {
    console.error('设置默认地址失败:', error);
    res.status(500).json({ success: false, error: '设置默认地址失
败', message: error.message });
  }
});

module.exports = router;

```

```
// ===== backend/routes/adminRoutes.js =====
```

```
const express = require('express');
const router = express.Router();
const { authenticateToken, requireAdmin, generateToken, ROLES } = require('../middleware/auth');
const db = require('../database.js');
const { ensureOrderPaymentColumns } = require('../services/orderPaymentSchema');
const { expireOldReviewOrders } = require('../utils/reviewExpire');
const { recordOrderIncome } = require('../services/incomeService');

async function hasOrderColumn(columnName) {
  const columns = await db.query(
    `SELECT COLUMN_NAME
     FROM INFORMATION_SCHEMA.COLUMNS
     WHERE TABLE_SCHEMA = DATABASE()
        AND TABLE_NAME = 'orders'
        AND COLUMN_NAME = ?`,
    [columnName]
  );
  return columns.length > 0;
}

async function ensureUnreadColumns() {
  const specs = [
    { name: 'user_unread', def: "ADD COLUMN user_unread TINYINT(1) NULL DEFAULT 0 COMMENT '用户未读标记: 1-有更新未查看' },
    { name: 'admin_unread', def: "ADD COLUMN admin_unread TINYINT(1) NULL DEFAULT 0 COMMENT '管理员未读标记: 1-有更新未查看' },
    { name: 'reject_reason', def: "ADD COLUMN reject_reason VARCHAR(255) NULL COMMENT '驳回/取消原因' }
  ];
  for (const s of specs) {
    try {
      const exists = await hasOrderColumn(s.name);
      if (exists) continue;
      await db.query(`ALTER TABLE orders ${s.def}`);
    } catch (e) {
      const dup = e && (e.code === 'ER_DUP_FIELDNAME' || e.errno === 1060 || String(e.message || '').includes('Duplicate column'));
      if (!dup) throw e;
    }
  }
}

async function trySetUnread(orderId, { user, admin } = {}) {
  await ensureUnreadColumns();
  const sets = [];
  const params = [];
  if (user !== undefined) { sets.push('user_unread = ?'); params.push(user ? 1 : 0); }
  if (admin !== undefined) { sets.push('admin_unread = ?'); params.push(admin ? 1 : 0); }
  if (sets.length === 0) return;
```

```

params.push(orderId);
const sql = `UPDATE orders SET ${sets.join(', ')}, updated_at = NOW() WHERE id = ?`;
try {
  await db.query(sql, params);
} catch (e) {
  const missing = e && (e.code === 'ER_BAD_FIELD_ERROR' || e.errno === 1054 || String(e.message || '').includes('Unknown column'));
  if (!missing) throw e;
  await ensureUnreadColumns();
  await db.query(sql, params);
}
}

async function ensureQuoteUnreadColumn() {
  const exists = await hasOrderColumn('quote_unread');
  if (exists) {
    return;
  }

  try {
    await db.query(
      `ALTER TABLE orders
      ADD COLUMN quote_unread TINYINT(1) NULL DEFAULT 0 COMMENT '用户是否有未读报价提醒: 0-已读, 1-未读'`
    );
  } catch (error) {
    const duplicateColumn = error && (error.code === 'ER_DUP_FIELDNAME' || error.errno === 1060 || String(error.message || '').includes('Duplicate column'));
    if (!duplicateColumn) {
      throw error;
    }
  }
}

async function ensureRepairReportFilesColumn() {
  try {
    await db.query(
      `ALTER TABLE orders
      ADD COLUMN repair_report_files JSON NULL COMMENT '维修报告返回文件列表'`
    );
  } catch (error) {
    const duplicateColumn = error && (error.code === 'ER_DUP_FIELDNAME' || error.errno === 1060 || String(error.message || '').includes('Duplicate column'));
    if (!duplicateColumn) {
      throw error;
    }
  }
}

```

```

function parseJsonFiles(value) {
  if (!value) return [];
  if (Array.isArray(value)) return value;
  if (typeof value === 'string') {
    try {
      return JSON.parse(value);
    } catch (e) {
      return [];
    }
  }
  return [];
}

```

```

async function submitQuoteWithUnreadFlag(params) {
  await ensureQuoteUnreadColumn();

```

```

  const sql = `UPDATE orders
    SET quote_price = ?,
        quote_description = ?,
        quote_files = ?,
        quote_status = 'pending',
        quote_unread = 1,
        quote_created_at = NOW(),
        quote_created_by = ?,
        status = 'quoted',
        assigned_to = ?,
        assigned_at = IF(assigned_to IS NULL OR assigned_to = 0, NOW(), assigned_at),
        updated_at = NOW()
  WHERE id = ?`;

```

```

  try {
    await db.query(sql, params);
  } catch (error) {
    const isMissingColumn = error && (error.code === 'ER_BAD_FIELD_ERROR' || error.errno === 1054 || String(
error.message || '').includes('Unknown column'));
    if (!isMissingColumn) {
      throw error;
    }
    await ensureQuoteUnreadColumn();
    await db.query(sql, params);
  }
}

```

```

/**
 * 管理员登录接口
 * POST /api/admin/login
 */
router.post('/login', async (req, res) => {
  try {
    const { phone } = req.body;

```

```
if (!phone) {
  return res.status(400).json({
    success: false,
    error: '手机号不能为空'
  });
}

// 查询用户
const users = await db.query(
  'SELECT * FROM users WHERE phone = ?',
  [phone]
);

if (users.length === 0) {
  return res.status(404).json({
    success: false,
    error: '用户不存在'
  });
}

const user = users[0];

// 检查用户状态
if (user.status !== 1) {
  return res.status(403).json({
    success: false,
    error: '用户已被禁用'
  });
}

// 验证是否为管理员或超级管理员
if (user.role !== ROLES.ADMIN && user.role !== ROLES.SUPER_ADMIN) {
  return res.status(403).json({
    success: false,
    error: '无管理员权限',
    currentRole: user.role
  });
}

// 更新最后登录时间
await db.query(
  'UPDATE users SET last_login_at = CURRENT_TIMESTAMP WHERE id = ?',
  [user.id]
).catch(() => {});

// 生成JWT令牌
const token = generateToken(user);

res.json({
```

```

us === 'completed' && Number(o.progress) === 100,
  `http=${comp.status}, status=${o.status}, progress=${o.progress}`);

// 用户查看已完成订单
const det2 = await api('GET', `/api/orders/${orderId}/detail`, user.token);
log(L('用户查看完成后详情'), det2.status === 200 && det2.data?.data?.status === 'completed',
  `status=${det2.data?.data?.status}`);
}

async function main() {
  console.log('===== E2E 全流程测试开始 =====\n');

  // 0. 健康检查
  const health = await api('GET', '/health');
  log('后端健康检查 /health', health.status === 200, JSON.stringify(health.data));
  const dbh = await api('GET', '/db-health');
  log('数据库健康检查 /db-health', dbh.status === 200, JSON.stringify(dbh.data));

  // 1. 准备测试账号
  const user = await ensureUser('e2e_test_user_openid', 'E2E测试用户', 'user');
  const admin = await ensureUser('e2e_test_admin_openid', 'E2E测试维修工', 'admin');
  log('准备测试账号', true, `user_id=${user.id}, admin_id=${admin.id}`);

  // 找一张测试图片
  const progressDir = path.join(__dirname, './uploads/progress');
  let imagePath = null;
  try {
    const dirs = fs.readdirSync(progressDir);
    outer: for (const d of dirs) {
      const sub = path.join(progressDir, d);
      if (fs.statSync(sub).isDirectory()) {
        for (const f of fs.readdirSync(sub)) {
          if (/\.(\.jpg|jpeg|png)$/i.test(f)) { imagePath = path.join(sub, f); break outer; }
        }
      }
    }
  } catch (e) {}

  console.log("\n----- 入口A: AI故障自检(diagnose) -> 下单 ---");
  const diag = await api('POST', '/api/diagnose/analyze', user.token, {
    deviceType: '手机', brand: '苹果', symptom: '无法开机', detail: '进水后无法开机'
  });
}

```

```

log('[A] AI故障自检 /diagnose/analyze', diag.status === 200
&& diag.data.success,
`http=${diag.status}, summary=${(diag.data?.data?.summary || "").slice(0, 40)}...`);

const orderA = await api('POST', '/api/orders/create', user.token, {
  orderType: 'repair', deviceType: '手机', deviceId: 1,
  problem: '无法开机', description: `AI自检结论: ${diag.data?.data?.summary || '疑似主板故障'}`,
  serviceType: 'shop', brand: '苹果', model: 'iPhone 15',
  estimatedPrice: 300, deviceCondition: '九成新'
});
const orderAId = orderA.data?.data?.order_id_numeric;
log('[A] 自检结论预填下单 /orders/create', orderA.status === 200
&& orderA.data.success && !!orderAId,
`http=${orderA.status}, orderId=${orderAId}, status=${orderA.data?.data?.order?.status}`);

console.log("\n----- 入口B: 扫码识别(scan) -> 下单 -----
-");
let scanOK = false, scanInfo = "";
if (imagePath) {
  const fd = new FormData();
  fd.append('image', new Blob([fs.readFileSync(imagePath)], { type: 'image/jpeg' } ), 'e2e_scan.jpg');
  const scan = await api('POST', '/api/scan/identify', user.token, fd);
  scanOK = scan.status === 200 && scan.data.success;
  scanInfo = `http=${scan.status}, ${JSON.stringify(scan.data?.data?.device_type_name || scan.data?.error
|| scan.data?.message || "").slice(0, 80)}`;
} else {
  scanInfo = '无测试图片';
}
log('[B] 扫码识别 /scan/identify', scanOK, scanInfo);

const orderB = await api('POST', '/api/orders/create', user.token, {
  orderType: 'repair', deviceType: '笔记本电脑', deviceId: 2,
  problem: '键盘失灵', description: '扫码识别设备后发起维修 (source: scan)',
  serviceType: 'home', brand: '联想', model: 'ThinkPad X1'
, estimatedPrice: 200, deviceCondition: '八成新',
  address: { contactName: 'E2E测试', contactPhone: '13800138000', province: '广东省', city: '深圳市', district: '南山区', detail: '科技园1号楼' }
});
const orderBId = orderB.data?.data?.order_id_numeric;
log('[B] 扫码预填下单 /orders/create', orderB.status === 200 &
& orderB.data.success && !!orderBId,
`http=${orderB.status}, orderId=${orderBId}`);

console.log("\n----- 入口C: 维修表单直接下单 -----");
const orderC = await api('POST', '/api/orders/create', user.token, {
  orderType: 'repair', deviceType: '手机', deviceId: 0, deviceTypeName: '游戏掌机',

```



```

    problem: '屏幕碎裂', description: '摔落导致屏幕碎裂, 触摸失灵',
    serviceType: 'shop', brand: '', model: 'Steam Deck', estimatedPrice: 0, deviceCondition: '七成新',
    isWaitingPrice: true
  });
  const orderCId = orderC.data?.data?.order_id_numeric;
  log('[C] 维修表单下单 /orders/create', orderC.status === 200 &
& orderC.data.success && !!orderCId,
    `http=${orderC.status}, orderId=${orderCId}, status=${orderC.data?.data?.order?.status}`);

// 用户订单列表可见
const list = await api('GET', '/api/orders/my?page=1&pageSize=20', user.token);
const listArr = list.data?.data?.orders || list.data?.data?.list || list.data?.data || [];
log('用户订单列表 /orders/my', list.status === 200 && list.data
a.success,
  `http=${list.status}, 数量=${Array.isArray(listArr) ? li
stArr.length : 'N/A'}`);

// 2. 完整生命周期
if (orderCId) {
  console.log(`\n----- 订单C 完整流程 (含拒绝报价分支 + 未支付守卫) --
-----`);
  await runLifecycle(orderCId, user, admin, '订单C', { tes
tReject: true, imagePath });
}
if (orderAId) {
  console.log(`\n----- 订单A 完整流程 (直接报价路径) -----`
);
  await runLifecycle(orderAId, user, admin, '订单A', { ima
gePath });
}
if (orderBId) {
  console.log(`\n----- 订单B 完整流程 (先接单再报价路径) -----
-`);
  await runLifecycle(orderBId, user, admin, '订单B', { acc
eptFirst: true, imagePath });
}

// 3. 汇总
console.log(`\n===== 测试汇总 =====`);
const pass = results.filter(r => r.ok).length;
const fail = results.filter(r => !r.ok);
console.log(`通过: ${pass}/${results.length}`);
if (fail.length) {
  console.log('失败项:');
  fail.forEach(f => console.log(` - ${f.step} | ${f.info}`));
}

// 4. 清理测试数据
console.log(`\n----- 清理测试数据 -----`);

```

```

const ids = [orderAId, orderBId, orderCId].filter(Boolean);
if (ids.length) {
  const ph = ids.map(() => '?').join(',');
  await db.query(`DELETE FROM order_progress_photos WHERE order_id IN (${ph})`, ids).catch(() => {});
  await db.query(`DELETE FROM transaction_income WHERE order_id IN (${ph})`, ids).catch(() => {});
  await db.query(`DELETE FROM orders WHERE id IN (${ph})`, ids).catch(() => {});
  for (const id of ids) {
    const dir = path.join(progressDir, String(id));
    if (fs.existsSync(dir)) fs.rmSync(dir, { recursive: true, force: true });
  }
}
await db.query(`DELETE FROM user_addresses WHERE user_id = ?`, [user.id]).catch(() => {});
await db.query(`DELETE FROM users WHERE openid IN ('e2e_test_user_openid', 'e2e_test_admin_openid')`).catch(() => {});
console.log('测试订单/账号/照片已清理');

process.exit(fail.length ? 1 : 0);
}

main().catch(e => { console.error('测试脚本异常:', e); process.exit(1); });

```

// ===== backend/test_scan_types.js =====

```

/**
 * 设备类型识别测试脚本（非侵入式：用 vm 沙箱加载 scanRoutes.js，不修改源码）
 * 覆盖全部 14 种支持类型，并重点验证「路由器被误判成手机」的修复场景。
 */
const fs = require('fs');
const path = require('path');
const vm = require('vm');

const filePath = path.join(__dirname, 'routes', 'scanRoutes.js');
const src = fs.readFileSync(filePath, 'utf8');

// ---- 构造沙箱，stub 掉 express / multer，真实加载 deviceData ----
const expressStub = { Router: () => ({ post: () => {}, get: () => {} }) };
const multerStub = function () { return { single: () => ({}) }; };
multerStub.memoryStorage = () => ({});

const sandbox = {
  require: (mod) => {
    if (mod === 'express') return expressStub;
    if (mod === 'multer') return multerStub;
    if (mod.startsWith('.')) return require(path.resolve(path.dirname(filePath), mod));
    return require(mod);
  },
  console,
  process,
  Buffer,
  module: { exports: {} },

```

```

    exports: {},
  };

  vm.createContext(sandbox);
  vm.runInContext(src, sandbox, { filename: filePath });

  const { matchDeviceByRules, detectHardCategory, normalizeDeviceType } = sandbox;

  function run(aiResult) {
    return matchDeviceByRules(aiResult);
  }

  // 每个测试用例：期望得到的设备大类
  const cases = [
    // 类型, 描述, aiResult
    ['手机', 'iPhone 15', { brand: 'apple', name: 'iPhone', type: '手机', features: { shape: '直板手机' } }],
    ['电脑/笔记本', '联想笔记本', { brand: '联想', name: 'Lenovo', type: '电脑/笔记本', features: { shape: '笔记本电脑' } }],
    ['平板', 'iPad', { brand: 'apple', name: 'iPad', type: '平板', features: { shape: '平板' } }],
    ['手表/手环', 'Apple Watch', { brand: 'apple', name: 'Apple Watch', type: '手表/手环', features: { shape: '智能手表' } }],
    ['耳机/音响', 'AirPods', { brand: 'apple', name: 'AirPods', type: '耳机/音响', features: { shape: 'twos耳机' } }],
    ['相机/摄像机', '佳能相机', { brand: '佳能', name: 'Canon', type: '相机/摄像机', features: { shape: '相机' } }],
    ['游戏机', 'Switch', { brand: '任天堂', name: 'Switch', type: '游戏机', features: { shape: '游戏机' } }],
    ['传感器/仪器', '西门子PLC', { brand: '西门子', name: 'Siemens', type: '传感器/仪器', features: { shape: '传感器' } }],
    ['无人机/航拍', '大疆无人机', { brand: '大疆', name: 'DJI', type: '无人机/航拍', features: { shape: '无人机' } }],
    ['智能家居', '小米智能音箱', { brand: '小米', name: '小米音箱', type: '智能家居', features: { shape: '智能音箱' } }],
    ['打印机/办公设备', 'HP 打印机', { brand: '惠普', name: 'HP', type: '打印机/办公设备', features: { shape: '打印机' } }],
    ['服务器', '戴尔机架服务器', { brand: '戴尔', name: 'PowerEdge', type: '服务器', features: { shape: '机架服务器' } }],
    ['路由器/网络设备', '华为路由器', { brand: '华为', name: '华为路由', type: '路由器/网络设备', features: { shape: '路由器' } }],
    ['显卡/电脑硬件', 'NVIDIA 显卡', { brand: 'nvidia', name: 'RTX', type: '显卡/电脑硬件', features: { shape: '显卡' } }],
  ];

  // 重点回归场景：AI 把「小米路由器」误判成手机 type=手机，但名称/描述含路由器+四天线
  const regressionCases = [
    ['路由器/网络设备', '小米路由器被AI误判为手机', { brand: '小米', name: '小米路由器', type: '手机', description: '白色小米路由器 四天线设计', features: { shape: '直板手机' } }],
  ];

```

```

    ['路由器/网络设备', '无品牌路由器(只识别到天线)', { brand: '', name: '路由器',
type: '未知', description: '白色路由器 四根外置天线', features: { shape: '路由器' } }],
    ['服务器', '无品牌机架服务器', { brand: '', name: '服务器', type: '未知',
description: '1U机架式服务器 多硬盘位', features: { shape: '机架服务器'
} }],
    ['显卡/电脑硬件', '无品牌显卡', { brand: '', name: '显卡', type: '未知',
description: '独立显卡 带风扇散热片', features: { shape: '显卡' } }],
    ,
];

```

```

let pass = 0, fail = 0;
function check(label, got, want) {
    const ok = got === want;
    ok ? pass++ : fail++;
    console.log(`${ok ? '□' : '□'} ${label}\n 期望: ${want}\n
n 实际: ${got}`);
    return ok;
}

```

```

console.log('===== 一、全部支持类型识别测试 =====');
console.log("");
for (const [want, desc, ai] of cases) {
    const r = run(ai);
    check(`[${desc}] -> ${want}`, r.type, want);
}

```

```

console.log("\n===== 二、关键回归场景（大方向锁定） =====");
for (const [want, desc, ai] of regressionCases) {
    const r = run(ai);
    check(`[${desc}] -> ${want}`, r.type, want);
}

```

```

console.log("\n===== 三、normalizeDeviceType 模糊归一化 =====");
const normCases = [
    ['手机', 'smartphone'], ['电脑/笔记本', 'laptop'], ['平板', 'iPad
tablet'],
    ['手表/手环', 'smartwatch'], ['耳机/音响', 'headphone'], ['相机/摄像机', 'camera'],
    ['游戏机', 'PlayStation'], ['传感器/仪器', 'PLC sensor'], ['无人机/航拍', 'drone'],
    ['智能家居', 'smart home'], ['打印机/办公设备', 'printer'], ['服务器', 'rack server'],
    ['路由器/网络设备', 'wifi router'], ['显卡/电脑硬件', 'NVIDIA GPU'],
];
for (const [want, raw] of normCases) {
    const got = normalizeDeviceType(raw);
    check(`normalize("${raw}") -> ${want}`, got, want);
}

```

```

console.log("\n===== 四、detectHardCategory 强特征命中 =====
=====');
const hardCases = [
  ['路由器/网络设备', '白色小米路由器 四天线设计'],
  ['服务器', '戴尔 PowerEdge 机架式服务器'],
  ['显卡/电脑硬件', 'NVIDIA RTX 4090 独立显卡'],
  [null, '苹果 iPhone 手机'],
];
for (const [want, blob] of hardCases) {
  const got = detectHardCategory(blob);
  check('detectHardCategory("${blob}") -> ${String(want)}', got, want);
}

console.log("\n===== 结果汇总 =====");
console.log('通过: ${pass} 失败: ${fail}');
process.exit(fail > 0 ? 1 : 0);

```

// ===== backend/verify_addr_update.js =====

```

// 用完整字段验证地址更新接口
const path = require('path');
require('dotenv').config({ path: path.join(__dirname, '.env'), override: true });
const jwt = require('jsonwebtoken');
const db = require('./database');

const BASE = 'http://localhost:3001';
const JWT_SECRET = process.env.JWT_SECRET;

async function main() {
  // 找测试用户的最新地址
  const addr = await db.query(
    'SELECT id, user_id FROM user_addresses ORDER BY id DESC LIMIT 1'
  );
  if (!addr.length) { console.log('无地址可测'); return process
.exit(0); }
  const user = await db.query('SELECT * FROM users WHERE id = ?', [addr[0].user_id]);
  const token = jwt.sign({ userId: user[0].id, openid: user[0].openid, nickname: user[0].nickname, role: use
r[0].role }, JWT_SECRET, { expiresIn: '1h' });

  // 完整字段更新
  const full = await fetch(BASE + `/api/addresses/${addr[0].id}`, {
    method: 'PUT',
    headers: { 'Authorization': 'Bearer ' + token, 'Content-Type': 'application/json' },
    body: JSON.stringify({
      contact_name: '完整字段更新', contact_phone: '13900000001'
    })
  });
  const res = await res.json();
  console.log('完整测试地址: ', res);
  // 完整测试地址: {
  //   id: 1,
  //   user_id: 1,
  //   province: '广东省', city: '深圳市', district: '南山区',
  //   detail_address: '完整测试地址', postal_code: '518000', tag
  // s: ['家', '公司'], is_default: true
  // }
}

```

```

});
const fullData = await full.json();
console.log(`[完整字段] http=${full.status} contact_name=${fullData.contact_name} tags=${JSON.stringify(fullData.tags)} is_default=${fullData.is_default}`);

// 验证 DB 实际更新
const dbRow = await db.query('SELECT * FROM user_addresses WHERE id = ?', [addr[0].id]);
console.log(`[DB核对] contact_name=${dbRow[0].contact_name} is_default=${dbRow[0].is_default} tags=${dbRow[0].tags}`);
);

process.exit(0);
}
main().catch(e => { console.error('异常:', e.message); process.exit(1); });

```

// ===== backend/verify_all_apis.js =====

```

/**
 * 综合功能验证脚本
 * 走一遍小程序后端前端(utils/api.js)调用的核心 API
 * 运行: node verify_all_apis.js
 */
const path = require('path');
require('dotenv').config({ path: path.join(__dirname, './.env') });
require('dotenv').config({ path: path.join(__dirname, '.env'), override: true });
const jwt = require('jsonwebtoken');
const db = require('./database');

const BASE = 'http://localhost:3001';
const results = [];
let USER_TOKEN, ADMIN_TOKEN, SUPER_TOKEN;
let TEST_USER_ID, TEST_DEVICE_ID, TEST_ADDRESS_ID, TEST_UNIT_ID, TEST_ORDER_ID;

function log(step, ok, info = '') {
  results.push({ step, ok, info });
  console.log(`${ok ? '[PASS]' : '[FAIL]'} ${step}${info ? ' | ' + info : ''}`);
}

async function api(method, url, token, body, raw = false) {
  const headers = {};
  if (token) headers['Authorization'] = 'Bearer ' + token;
  const opts = { method, headers };
  if (body !== undefined) {
    if (body instanceof FormData) {
      opts.body = body;
    } else {
      headers['Content-Type'] = 'application/json';
      opts.body = JSON.stringify(body);
    }
  }
}

```

```

}
const res = await fetch(BASE + url, opts);
let data = {};
if (!raw) { try { data = await res.json(); } catch (e) {} }
else { data = await res.text(); }
return { status: res.status, data };
}

function makeToken(user) {
  return jwt.sign({ userId: user.id, openid: user.openid, nickname: user.nickname, role: user.role },
    process.env.JWT_SECRET, { expiresIn: '1h' });
}

async function ensureUser(openid, nickname, role, phone) {
  const rows = await db.query('SELECT id FROM users WHERE openid = ?', [openid]);
  let id;
  if (rows.length) {
    id = rows[0].id;
    await db.query('UPDATE users SET role = ?, status = 1, phone = ? WHERE id = ?', [role, phone || null, id]);
  } else {
    const r = await db.query(
      `INSERT INTO users (openid, nickname, role, status, phone) VALUES (?, ?, ?, 1, ?)`
      [openid, nickname, role, phone || null]
    );
    id = r.insertId;
  }
  const user = await db.query('SELECT * FROM users WHERE id = ?', [id]);
  return user[0];
}

async function main() {
  console.log('===== 综合功能验证开始 =====\n');

  // 0. 健康检查
  const health = await api('GET', '/health');
  log('后端健康检查', health.status === 200, JSON.stringify(health.data));
  const dbh = await api('GET', '/db-health');
  log('数据库健康检查', dbh.status === 200 && dbh.data.status === 'OK', JSON.stringify(dbh.data));

  // 1. 管理员登录（真实接口）
  console.log('\n---- 1. 管理员登录 ----');
  const login = await api('POST', '/api/admin/login', null, { phone: '13800138000' });
  ADMIN_TOKEN = login.data?.token;
  log('超级管理员登录(13800138000)', login.status === 200 && !!ADMIN_TOKEN && login.data.user?.role === 'super_admin',
    `role=${login.data?.user?.role}`);
  const login2 = await api('POST', '/api/admin/login', null, { phone: '13800138001' });

```

```

const admin2Token = login2.data?.token;
log('普通管理员登录(13800138001)', login2.status === 200 && !!admin2Token, `role=${login2.data?.user?.role}`);

// 准备测试用户
const testUser = await ensureUser('verify_test_user', '验证测试用户', 'user', '139000000001');
TEST_USER_ID = testUser.id;
USER_TOKEN = makeToken(testUser);
log('准备测试用户', !!USER_TOKEN, `id=${TEST_USER_ID}`);
const adminUser = await ensureUser('verify_test_admin', '验证测试维修工', 'admin', '139000000002');
const adminToken = makeToken(adminUser);
log('准备测试维修工', !!adminToken, `id=${adminUser.id}`);

// 2. 用户模块
console.log(`\n---- 2. 用户模块 ----`);
const uinfo = await api('GET', '/api/user/info', USER_TOKEN);
log('获取用户信息', uinfo.status === 200, `id=${uinfo.data?.id}, role=${uinfo.data?.role}`);
const upinfo = await api('PUT', '/api/user/info', USER_TOKEN, { nickname: '验证测试用户-改', phone: '13900000001' });
log('更新用户信息', upinfo.status === 200, `nickname=${upinfo.data?.nickname}`);
// 非法手机号应拒绝
const badPhone = await api('PUT', '/api/user/info', USER_TOKEN, { phone: '123' });
log('非法手机号校验', badPhone.status === 400, `http=${badPhone.status}`);

// 3. 地址模块
console.log(`\n---- 3. 地址模块 ----`);
const addrCreate = await api('POST', '/api/addresses', USER_TOKEN, { contact_name: '测试联系人', contact_phone: '13900000001', province: '广东省', city: '深圳市', district: '南山区', detail_address: '科技园测试路1号' });
TEST_ADDRESS_ID = addrCreate.data?.id;
log('创建地址', addrCreate.status === 200 && !!TEST_ADDRESS_ID, `id=${TEST_ADDRESS_ID}`);
const addrList = await api('GET', '/api/addresses', USER_TOKEN);
log('获取地址列表', addrList.status === 200 && Array.isArray(addrList.data), `count=${Array.isArray(addrList.data) ? addrList.data.length : 'N/A'}`);
if (TEST_ADDRESS_ID) {
  const addrUpd = await api('PUT', `/api/addresses/${TEST_ADDRESS_ID}`, USER_TOKEN, { contact_name: '测试联系人改' });
  log('更新地址', addrUpd.status === 200, `http=${addrUpd.status}`);
  const addrDefault = await api('POST', `/api/addresses/${TEST_ADDRESS_ID}/default`, USER_TOKEN, {});
  log('设置默认地址', addrDefault.status === 200, `http=${addrDefault.status}`);
}

```



```

}

// 4. 用户设备模块
console.log("\n---- 4. 用户设备模块 ----");
const devCreate = await api('POST', '/api/user-devices', USER_TOKEN,
  { device_type_id: 1, device_type_name: '手机', brand_name: '苹果', device_model: 'iPhone 15', device_nickname: '我的iPhone', device_purpose: 'repair' });
TEST_DEVICE_ID = devCreate.data?.data?.id;
log('创建设备', devCreate.status === 201 && !TEST_DEVICE_ID, `id=${TEST_DEVICE_ID}`);
const devList = await api('GET', '/api/user-devices', USER_TOKEN);
log('获取设备列表', devList.status === 200, `count=${Array.isArray(devList.data?.data) ? devList.data.data.length : 'N/A'}`);
if (TEST_DEVICE_ID) {
  const devDetail = await api('GET', `/api/user-devices/${TEST_DEVICE_ID}`, USER_TOKEN);
  log('获取设备详情', devDetail.status === 200, `http=${devDetail.status}`);
  const devDefault = await api('POST', `/api/user-devices/${TEST_DEVICE_ID}/default`, USER_TOKEN, {});
  log('设置默认设备', devDefault.status === 200, `http=${devDefault.status}`);
}

// 5. 单位模块
console.log("\n---- 5. 单位模块 ----");
const unitCreate = await api('POST', '/api/units', USER_TOKEN,
  { name: '测试单位', address: '深圳市南山区', contact_name: '单位联系人', contact_phone: '13900000001' });
TEST_UNIT_ID = unitCreate.data?.id;
log('创建单位', unitCreate.status === 200 && !TEST_UNIT_ID, `id=${TEST_UNIT_ID}`);
const unitList = await api('GET', '/api/units', USER_TOKEN);
log('获取单位列表', unitList.status === 200, `http=${unitList.status}`);

// 6. 产品模块
console.log("\n---- 6. 产品模块 ----");
const prodList = await api('GET', '/api/products', USER_TOKEN);
log('获取产品列表', prodList.status === 200, `count=${Array.isArray(prodList.data?.data) ? prodList.data.data.length : (prodList.data?.data?.products?.length ?? 'N/A')}`);

// 7. AI诊断 + 扫码
console.log("\n---- 7. AI诊断 / 扫码 ----");
const diag = await api('POST', '/api/diagnose/analyze', USER_TOKEN,
  { deviceType: '手机', brand: '华为', symptom: '屏幕碎了', details: '摔落屏幕碎裂' });
log('AI故障诊断', diag.status === 200, `http=${diag.status}, summary=${(diag.data?.data?.summary || '').slice(0, 30)}`);

```

```

);

// 8. 订单创建 + 查询
console.log("\n---- 8. 订单模块 ----");
const orderCreate = await api('POST', '/api/orders/create', USER_TOKEN, {
  orderType: 'repair', deviceType: '手机', deviceId: TEST_
DEVICE_ID || 0,
  problem: '屏幕碎裂', description: '验证脚本下单',
  serviceType: 'shop', brand: '苹果', model: 'iPhone 15',
estimatedPrice: 300,
  deviceCondition: '九成新', addressId: TEST_ADDRESS_ID
});
TEST_ORDER_ID = orderCreate.data?.data?.order_id_numeric || orderCreate.data?.data?.order?.id || orderCrea
te.data?.data?.id;
log('创建维修订单', orderCreate.status === 200 && !!TEST_ORDER
_ID, `orderId=${TEST_ORDER_ID}`);
if (TEST_ORDER_ID) {
  const orderList = await api('GET', `/api/orders/user/${TEST_USER_ID}?page=1&pageSize=20`, USER_TOKEN);
  log('获取用户订单列表', orderList.status === 200 && orderList.
data?.success, `http=${orderList.status}`);
  const orderDetail = await api('GET', `/api/orders/${TEST_ORDER_ID}/detail`, USER_TOKEN);
  log('获取订单详情', orderDetail.status === 200 && orderDetai
l.data?.data?.order?.status, `status=${orderDetail.data?.d
ata?.order?.status}`);
  // 管理员报价
  const quote = await api('PUT', `/api/admin/orders/${TEST_ORDER_ID}/quote`, ADMIN_TOKEN,
    { quote_price: 299, quote_description: '综合验证报价' });
  log('管理员报价', quote.status === 200, `http=${quote.statu
s}`);
  // 用户确认报价
  const acceptQ = await api('PUT', `/api/orders/${TEST_ORDER_ID}/accept-quote`, USER_TOKEN);
  log('用户确认报价', acceptQ.status === 200, `http=${acceptQ.
status}`);
  // 管理员接单/处理
  const process = await api('PUT', `/api/admin/orders/${TEST_ORDER_ID}/process`, ADMIN_TOKEN);
  log('管理员开始处理(未支付应被拒)', process.status === 400, `http=$
${process.status}, msg=${process.data?.error || process.dat
a?.message}`);
}

// 9. 管理员功能
console.log("\n---- 9. 管理员功能 ----");
const dash = await api('GET', '/api/admin/dashboard-stats', ADMIN_TOKEN);
log('仪表盘统计', dash.status === 200, `http=${dash.status}`)
;
const allOrders = await api('GET', '/api/admin/all-orders?page=1&pageSize=10', ADMIN_TOKEN);
log('管理员全部订单', allOrders.status === 200, `http=${allOrde
rs.status}, total=${allOrders.data?.data?.total ?? allOrde
rs.data?.total}`);
const pending = await api('GET', '/api/admin/pending-count', ADMIN_TOKEN);

```

```

    log('待处理计数', pending.status === 200, `http=${pending.status}`);
    const admins = await api('GET', '/api/admin/admins', ADMIN_TOKEN);
    log('管理员列表', admins.status === 200, `count=${Array.isArray(admins.data?.data) ? admins.data.data.length : 'N/A'}`);
};

// 10. 超级管理员功能 (若 super admin)
console.log("\n---- 10. 超级管理员 ----");
const superList = await api('GET', '/api/super-admin/users', ADMIN_TOKEN).catch(e => ({ status: 0 }));
if (superList.status === 200 || superList.status > 0) {
    log('超级管理员-用户列表', superList.status === 200, `http=${superList.status}`);
} else {
    log('超级管理员-用户列表', false, '请求异常');
}

// 11. 进度申请
console.log("\n---- 11. 进度申请 ----");
const applyCreate = await api('POST', '/api/progress-apply', USER_TOKEN, {
    order_id: TEST_ORDER_ID, customer_name: '验证测试用户', phone: '13900000001', device_name: 'iPhone 15', device_model: 'iPhone 15', progress_type: 'query', apply_reason: '想了解维修情况', expected_time: '2026-08-05' });
log('创建进度申请', applyCreate.status === 201 || applyCreate.status === 200, `http=${applyCreate.status}, ${JSON.stringify(applyCreate.data?.error || applyCreate.data?.message || '')}`);
const applyList = await api('GET', '/api/progress-apply/my/list', USER_TOKEN);
log('我的进度申请列表', applyList.status === 200, `http=${applyList.status}`);

// 12. 售后
console.log("\n---- 12. 售后 ----");
if (TEST_DEVICE_ID) {
    const aft = await api('GET', `/api/after-sales/device/${TEST_DEVICE_ID}`, USER_TOKEN);
    log('设备售后总览', aft.status === 200, `http=${aft.status}`);
};

// 13. 知识库
console.log("\n---- 13. 知识库 ----");
const kbCats = await api('GET', '/api/knowledge/categories', USER_TOKEN);
log('知识库分类', kbCats.status === 200, `http=${kbCats.status}`);

// 14. 位置服务
console.log("\n---- 14. 位置服务 ----");
const geo = await api('GET', '/api/location/ip-location', USER_TOKEN);
log('IP定位', geo.status === 200, `http=${geo.status}`);

```

```

// 15. 产品搜索
console.log("\n---- 15. 其他 ----");
const prodSearch = await api('GET', '/api/products/search/h/手机', USER_TOKEN);
log('产品搜索', prodSearch.status === 200, `http=${prodSearch.status}`);

// 汇总
console.log("\n===== 验证汇总 =====");
const pass = results.filter(r => r.ok).length;
const fail = results.filter(r => !r.ok);
console.log(`通过: ${pass}/${results.length}`);
if (fail.length) {
  console.log('失败项:');
  fail.forEach(f => console.log(` - ${f.step} | ${f.info}`));
}
process.exit(fail.length ? 1 : 0);
}

main().catch(e => { console.error('脚本异常:', e); process.exit(1); });

// ===== backend/verify_aux.js =====
/**
 * 辅助模块冒烟测试: 维修记录 / 售后申请 / 配送 / 回收估价 / 管理员维修报告
 * 运行: node verify_aux.js
 */
const fs = require('fs');
const path = require('path');
const mysql = require('mysql2/promise');
const jwt = require('jsonwebtoken');
require('dotenv').config({ path: path.join(__dirname, './.env') });
require('dotenv').config({ path: path.join(__dirname, '.env'), override: true });
const BASE = 'http://localhost:3001';
const results = [];
function log(step, ok, info = '') { results.push({ step, ok, info }); console.log(`${ok ? '[PASS]' : '[FAIL]'} ${step}${info ? ' | ' + info : ''}`); }
async function api(m, u, t, b) {
  const h = {}; if (t) h['Authorization'] = 'Bearer ' + t;
  const o = { method: m, headers: h };
  if (b instanceof FormData) o.body = b; else if (b !== undefined) { h['Content-Type'] = 'application/json'; o.body = JSON.stringify(b); }
  const r = await fetch(BASE + u, o); let d = {}; try { d = await r.json(); } catch (e) {}
  return { status: r.status, data: d };
}

const pool = mysql.createPool({ host: process.env.DB_HOST, port: parseInt(process.env.DB_PORT) || 3306, user: process.env.DB_USER, password: process.env.DB_PASSWORD, database: process.env.DB_NAME, connectionLimit: 5, charset: 'utf8mb4' });
async function ensureUser(openid, nickname, role) {

```

```

const c = await pool.getConnection();
const [rows] = await c.query('SELECT id FROM users WHERE openid = ?', [openid]);
let id;
if (rows.length) { id = rows[0].id; await c.query('UPDATE users SET role=?, status=1 WHERE id=?', [role, id]); }
else { const r = await c.query('INSERT INTO users (openid,nickname,role,status,created_at) VALUES (?,?,,1,NOW())', [openid, nickname, role]); id = r[0].insertId; }
c.release();
return { id, token: jwt.sign({ userId: id, openid, nickname, role }, process.env.JWT_SECRET, { expiresIn: '2h' }) };
}

async function getOrder(id) { const [r] = await pool.query('SELECT status, assigned_to, delivery_status FROM orders WHERE id=?', [id]); return r[0]; }

(async () => {
  console.log('===== 辅助模块冒烟测试开始 =====\n');
  const U = 'aux_user_' + Date.now(), A = 'aux_admin_' + Date.now();
  const user = await ensureUser(U, '辅助验证用户', 'user');
  const admin = await ensureUser(A, '辅助验证管理员', 'admin');

  // 造一个已完成订单
  const o = await api('POST', '/api/orders/create', user.token, { orderType: 'repair', deviceType: 1, deviceId: 1, problem: '屏幕碎裂', description: '辅助测试', serviceType: 'shop', brand: '苹果', model: 'iPhone 15' });
  const old = o.data?.data?.order_id_numeric;
  await api('PUT', `/api/admin/orders/${old}/quote`, admin.token, { quote_price: 100, quote_description: '换屏' });
  await api('PUT', `/api/orders/${old}/accept-quote`, user.token);
  await pool.query("UPDATE orders SET payment_status='paid', paid_at=NOW(), updated_at=NOW() WHERE id=?", [old]);
  await api('PUT', `/api/admin/orders/${old}/process`, admin.token);
  await api('PUT', `/api/admin/orders/${old}/complete`, admin.token);
  let od = await getOrder(old);
  log('前置-完成订单', od.status === 'completed', `status=${od.status}`);

  // 管理员把订单分配给自己（维修记录需要 assigned_to=自己）
  const assign = await api('PUT', `/api/admin/orders/${old}/assign`, admin.token, { adminId: admin.id });
  log('管理员分配订单 assign', assign.status === 200, `http=${assign.status}, ${JSON.stringify(assign.data).slice(0,60)}`);

  // 1. 维修记录
  const rr = await api('POST', '/api/repair-records', admin.token, { order_id: old, stage: '拆机检测', title: '检测主板故障', description: '发现进水腐蚀' });
  const rrId = rr.data?.data?.id;
  log('维修记录-新增 /repair-records', rr.status === 200 && rr.data?.success && !rrId, `http=${rr.status}, id=${rrId}`);
  const rrList = await api('GET', `/api/repair-records/order/${old}`, user.token);

```

```
log('维修记录-按订单查询', rrList.status === 200 && Array.isArray(
rrList.data?.data) && rrList.data.data.length > 0, `http=
${rrList.status}, 条数=${rrList.data?.data?.length}`);
```

// 2. 售后申请

```
const as = await api('POST', '/api/after-sales/request',
user.token, { order_id: old, product_name: 'iPhone 15', p
roduct_model: 'iPhone 15', type: 'repair', description: '换
屏后触控失灵', contact_phone: '13800138000' });
const asId = as.data?.data?.id;
log('售后-申请 /after-sales/request', as.status === 200 && a
s.data?.success && !!asId, `http=${as.status}, id=${asId}`
);
const asMy = await api('GET', '/api/after-sales/my', user.token);
log('售后-我的申请列表', asMy.status === 200, `http=${asMy.statu
s}, ${Array.isArray(asMy.data?.data) ? '条数=' + asMy.data.dat
a.length : JSON.stringify(asMy.data).slice(0,40)}`);
```

// 3. 配送

```
const dp = await api('POST', '/api/delivery/persons', ad
min.token, { user_id: admin.id, name: '配送员甲', phone: '1370
0137000' });
const dpId = dp.data?.data?.id;
log('配送-新增配送员 /delivery/persons', dp.status === 200 && d
p.data?.success && !!dpId, `http=${dp.status}, id=${dpId}`
);
const dpList = await api('GET', '/api/delivery/persons', admin.token);
log('配送-配送员列表', dpList.status === 200 && dpList.data?.su
ccess, `http=${dpList.status}`);
const dAssign = await api('PUT', `/api/delivery/orders/${old}/assign`, admin.token, { delivery_person_id:
dpId, delivery_fee: 10 });
log('配送-分配订单', dAssign.status === 200 && dAssign.data?.s
uccess, `http=${dAssign.status}, ${JSON.stringify(dAssign.
data).slice(0,60)}`);
const dStatus = await api('PUT', `/api/delivery/orders/${
old}/status`, admin.token, { status: 'assigned', tracking
_number: 'SF123', delivery_notes: '已取件' });
log('配送-更新状态 assigned', dStatus.status === 200 && dStatu
s.data?.success, `http=${dStatus.status}, ${JSON.stringify
(dStatus.data).slice(0,60)}`);
```

// 4. 管理员维修报告

```
const rep = await api('PUT', `/api/admin/orders/${old}/r
epair-report`, admin.token, { report: '已更换屏幕总成, 功能正常', file
s: [] });
log('管理员-维修报告 repair-report', rep.status === 200 && rep.
data?.success, `http=${rep.status}`);
```

// 5. 回收估价 (开放接口, 依赖Deepseek, 尽力而为)

```
const rec = await api('POST', '/api/recycle/evaluate', n
```

```

ull, { product: { category: '手机', brand: '苹果', model: 'iPhone 13', basePrice: 3000, specs: '256G' }, answers: { screen: { label: '完好' }, body: { label: '轻微划痕' } } });
  log('回收估价 /recycle/evaluate', rec.status === 200, `http=${rec.status}, success=${rec.data?.success}, price=${rec.data?.data?.price || rec.data?.message || ''}`);

// 汇总
console.log(`\n===== 辅助模块汇总 =====`);
const pass = results.filter(r => r.ok).length;
const fail = results.filter(r => !r.ok);
console.log(`通过: ${pass}/${results.length}`);
if (fail.length) { console.log('失败项:'); fail.forEach(f => console.log(` - ${f.step} | ${f.info}`)); }

// 清理
console.log(`\n----- 清理 -----`);
await pool.query('DELETE FROM repair_records WHERE order_id=?', [old]).catch(()=>{});
await pool.query('DELETE FROM after_sales_requests WHERE order_id=?', [old]).catch(()=>{});
await pool.query('DELETE FROM delivery_persons WHERE user_id=?', [admin.id]).catch(()=>{});
await pool.query('DELETE FROM orders WHERE id=?', [old]).catch(()=>{});
await pool.query('DELETE FROM users WHERE openid IN (?,?)', [U, A]).catch(()=>{});
await pool.end();
process.exit(fail.length ? 1 : 0);
})();catch(e => { console.error('异常:', e); process.exit(1); });

// ===== backend/verify_full_flow.js =====
/**
 * 全功能流程验证（修正版）：覆盖 地址/单位/下单(到店+上门)/真实订单列表/
 * 报价-拒绝-重报-接受/支付守卫/开始处理/进度上报/进度照片落盘/完成/评价/取消/未读计数
 * 运行: node verify_full_flow.js （默认打向 http://localhost:3001）
 */
const fs = require('fs');
const path = require('path');
const mysql = require('mysql2/promise');
const jwt = require('jsonwebtoken');
require('dotenv').config({ path: path.join(__dirname, './.env') });
require('dotenv').config({ path: path.join(__dirname, '.env'), override: true });

const BASE = process.env.TEST_BASE || 'http://localhost:3001';
const results = [];
function log(step, ok, info = '') {
  results.push({ step, ok, info });
  console.log(`${ok ? '[PASS]' : '[FAIL]'} ${step}${info ? ' | ' + info : ''}`);
}

async function api(method, url, token, body) {
  const headers = {};

```

```

if (token) headers['Authorization'] = 'Bearer ' + token;
const opts = { method, headers };
if (body instanceof FormData) {
  opts.body = body;
} else if (body !== undefined) {
  headers['Content-Type'] = 'application/json';
  opts.body = JSON.stringify(body);
}
const res = await fetch(BASE + url, opts);
let data = {};
try { data = await res.json(); } catch (e) {}
return { status: res.status, data };
}

```

```

async function ensureUser(openid, nickname, role) {
  const c = await pool.getConnection();
  const [rows] = await c.query('SELECT id FROM users WHERE openid = ?', [openid]);
  let id;
  if (rows.length) {
    id = rows[0].id;
    await c.query('UPDATE users SET role = ?, status = 1 WHERE id = ?', [role, id]);
  } else {
    const r = await c.query(
      'INSERT INTO users (openid, nickname, role, status, created_at) VALUES (?, ?, ?, 1, NOW())',
      [openid, nickname, role]
    );
    id = r[0].insertId;
  }
  c.release();
  const token = jwt.sign({ userId: id, openid, nickname, role }, process.env.JWT_SECRET, { expiresIn: '2h' });
  return { id, token };
}

```

```

async function getOrder(id) {
  const [rows] = await pool.query(
    'SELECT id, status, quote_status, quote_price, payment_status, pay_amount, progress, assigned_to FROM orders WHERE id = ?',
    [id]
  );
  return rows[0];
}

```

```

const pool = mysql.createPool({
  host: process.env.DB_HOST, port: parseInt(process.env.DB_PORT) || 3306,
  user: process.env.DB_USER, password: process.env.DB_PASSWORD, database: process.env.DB_NAME,
  connectionLimit: 5, charset: 'utf8mb4'
});

```

```

async function main() {

```



```

console.log('===== 全功能流程验证开始 (BASE=' + BASE + ') ==
=====\\n');
const U = 'verify_user_' + Date.now();
const A = 'verify_admin_' + Date.now();

// 0. 健康
const h = await api('GET', '/health');
log('健康检查 /health', h.status === 200, JSON.stringify(h.data));
const dh = await api('GET', '/db-health');
log('数据库健康 /db-health', dh.status === 200, JSON.stringify(dh.data));

const user = await ensureUser(U, '验证用户', 'user');
const admin = await ensureUser(A, '验证维修工', 'admin');
log('准备测试账号', true, `user_id=${user.id}, admin_id=${admin.id}`);

// 1. 地址管理
const addr = await api('POST', '/api/addresses', user.token, {
  contact_name: '验证收货人', contact_phone: '13800138000',
  province: '广东省', city: '深圳市', district: '南山区', detail_address: '科技园验证楼1号', is_default: true
});
const addrId = addr.data?.id;
log('地址-新增 /api/addresses', addr.status === 200 && !!addrId, `http=${addr.status}, id=${addrId}`);
const addrList = await api('GET', '/api/addresses', user.token);
log('地址-列表 /api/addresses', addrList.status === 200, `http=${addrList.status}, 数量=${Array.isArray(addrList.data) ? addrList.data.length : JSON.stringify(addrList.data).slice(0,40)}`);
if (addrId) {
  const df = await api('POST', `/api/addresses/${addrId}/default`, user.token);
  const up = await api('PUT', `/api/addresses/${addrId}`, user.token, { contact_name: '验证收货人改', contact_phone: '13800138000', province: '广东省', city: '深圳市', district: '南山区', detail_address: '科技园验证楼2号' });
  log('地址-设默认/编辑', df.status === 200 && up.status === 200, `default=${df.status}, update=${up.status}`);
}

// 2. 单位管理
const unit = await api('POST', '/api/units', user.token, {
  name: '验证单位公司', address: '验证路1号', contact_name: '李经理', contact_phone: '13900139000', is_default: true
});
const unitId = unit.data?.id;
log('单位-新增 /api/units', unit.status === 200 && !!unitId, `http=${unit.status}, id=${unitId}`);

```

```

const unitList = await api('GET', '/api/units', user.token);
log('单位-列表 /api/units', unitList.status === 200, `http=${unitList.status}, 数量=${Array.isArray(unitList.data) ? unitList.data.length : JSON.stringify(unitList.data).slice(0, 40)}`);

// 3. 下单: 到店 (无地址)
const oShop = await api('POST', '/api/orders/create', user.token, {
  orderType: 'repair', deviceType: 1, deviceId: 1, problem: '无法开机',
  description: '进水后无法开机', serviceType: 'shop', brand: '苹果', model: 'iPhone 15', estimatedPrice: 300, deviceCondition: '九成新'
});
const oShopId = oShop.data?.data?.order_id_numeric;
log('下单-到店 /orders/create', oShop.status === 200 && oShop.data?.success && !!oShopId, `http=${oShop.status}, id=${oShopId}, status=${oShop.data?.data?.order?.status}`);

// 4. 下单: 上门 (带地址ID)
const oHome = await api('POST', '/api/orders/create', user.token, {
  orderType: 'repair', deviceType: 2, deviceId: 2, problem: '键盘失灵',
  description: '部分按键失灵', serviceType: 'home', brand: '联想', model: 'ThinkPad X1', estimatedPrice: 200, deviceCondition: '八成新', addressId: addrId
});
const oHomeId = oHome.data?.data?.order_id_numeric;
log('下单-上门(带地址) /orders/create', oHome.status === 200 && oHome.data?.success && !!oHomeId, `http=${oHome.status}, id=${oHomeId}`);
if (oHomeId) {
  const det = await api('GET', `/api/orders/${oHomeId}/detail`, user.token);
  const od = det.data?.data?.order;
  log('下单-上门地址落库校验', !!od && od.address_id && od.contact_name === '验证收货人改', `address_id=${od?.address_id}, contact=${od?.contact_name}`);
}

// 5. 真实"我的订单"列表 (前端 api.js 用的是 /orders/user/:userId)
const mylist = await api('GET', `/api/orders/user/${user.id}`, user.token);
const arr = mylist.data?.data?.orders || mylist.data?.data?.list || mylist.data?.data || [];
const hasShop = Array.isArray(arr) && arr.some(o => o.id === oShopId);
const hasHome = Array.isArray(arr) && arr.some(o => o.id === oHomeId);
log('我的订单列表 /orders/user/:userId', mylist.status === 200 && hasShop && hasHome, `http=${mylist.status}, 数量=${Array.isArray(arr) ? arr.length : 'N/A'}`);

// 6. 完整生命周期 (含 拒绝->重报) on oShop
if (oShopId) {

```

```

    const q = await api('PUT', `/api/admin/orders/${oShopId}/quote`, admin.token, { quote_price: 199.5, quote_description: '更换故障部件' });
    let o = await getOrder(oShopId);
    log('管理员报价 quote', q.status === 200 && o.status === 'quoted' && o.quote_status === 'pending', `status=${o.status}, quote_status=${o.quote_status}`);

    const det = await api('GET', `/api/orders/${oShopId}/detail`, user.token);
    const od = det.data?.data?.order;
    log('用户查看详情(正确路径 data.data.order.status)', det.status === 200 && od && od.status === 'quoted' && od.quote_price === 199.5, `status=${od?.status}, quote_price=${od?.quote_price}`);

    const rej = await api('PUT', `/api/orders/${oShopId}/reject-quote`, user.token, { reason: '价格偏高' });
    o = await getOrder(oShopId);
    log('用户拒绝报价 reject-quote', rej.status === 200 && o.status === 'pending' && o.quote_status === 'rejected', `status=${o.status}, quote_status=${o.quote_status}`);

    const q2 = await api('PUT', `/api/admin/orders/${oShopId}/quote`, admin.token, { quote_price: 149.5, quote_description: '优惠价重报' });
    o = await getOrder(oShopId);
    log('管理员重报(二次报价)', q2.status === 200 && o.status === 'quoted' && o.quote_price === 149.5, `status=${o.status}, quote_price=${o.quote_price}`);

    const acc = await api('PUT', `/api/orders/${oShopId}/accept-quote`, user.token);
    o = await getOrder(oShopId);
    log('用户接受报价 accept-quote', acc.status === 200 && o.status === 'confirmed' && o.quote_status === 'accepted' && o.payment_status === 'unpaid', `status=${o.status}, payment_status=${o.payment_status}`);

    const guard = await api('PUT', `/api/admin/orders/${oShopId}/process`, admin.token);
    log('未支付禁止开始处理(守卫)', guard.status === 400, `http=${guard.status}, msg=${guard.data?.error}`);

    await pool.query("UPDATE orders SET payment_status='paid', paid_at=NOW(), updated_at=NOW() WHERE id=?", [oShopId]);
    const p1 = await api('PUT', `/api/admin/orders/${oShopId}/process`, admin.token);
    o = await getOrder(oShopId);
    log('管理员开始处理 process', p1.status === 200 && o.status === 'processing', `status=${o.status}`);

    const pr1 = await api('PUT', `/api/orders/${oShopId}/progress`, admin.token, { progress: 30 });
    const pr2 = await api('PUT', `/api/orders/${oShopId}/progress`, admin.token, { progress: 70 });

```

```

    o = await getOrder(oShopId);
    log('维修进度上报 30->70', pr1.status === 200 && pr2.status
=== 200 && Number(o.progress) === 70, `progress=${o.progre
ss}`);

    // 进度照片上传 (真实文件, 校验落盘)
    const imgPath = 'D:/maintain/电子维修2.0/uploads/progress/
25/1779522114199-38925454.jpg';
    let uploadOk = false, savedPath = '';
    if (fs.existsSync(imgPath)) {
        const fd = new FormData();
        fd.append('orderId', String(oShopId));
        fd.append('description', '维修进度反馈: 部件已更换');
        fd.append('images', new Blob([fs.readFileSync(imgPath)], { type: 'image/jpeg' })), 'verify_progress.jpg
');
        const up = await api('POST', '/api/progress/photos/upload', admin.token, fd);
        const imgs = up.data?.data?.images || up.data?.images || [];
        uploadOk = up.status === 200 && imgs.length > 0;
        savedPath = Array.isArray(imgs) ? imgs[0] : '';
        log('进度照片上传(校验落盘)', uploadOk, `http=${up.status}, fi
les=${JSON.stringify(imgs).slice(0,80)}`);
        if (savedPath) {
            const localFile = path.join(__dirname, '..', savedPath.replace(/^\/+/, ''));
            const onDisk = fs.existsSync(localFile);
            log(' 文件真实落盘校验', onDisk, `path=${localFile}`);
        }
    } else {
        log('进度照片上传(校验落盘)', false, '未找到测试图片');
    }

    const fb = await api('GET', `/api/progress/feedbacks/${oShopId}`, user.token);
    const fbList = fb.data?.data || fb.data?.data?.feedbacks || [];
    log('用户查看进度反馈列表', fb.status === 200 && fb.data?.succes
s, `http=${fb.status}, 条数=${Array.isArray(fbList) ? fbList
.length : 'N/A'}`);

    const unread = await api('GET', '/api/orders/progress-unread-count', user.token);
    log('未读进度提醒计数', unread.status === 200 && unread.data?.
success, `count=${JSON.stringify(unread.data?.data)}`);

    const read = await api('PUT', `/api/orders/${oShopId}/progress-read`, user.token);
    log('标记进度已读', read.status === 200, `http=${read.status
}`);

    const comp = await api('PUT', `/api/admin/orders/${oShopId}/complete`, admin.token);
    o = await getOrder(oShopId);
    log('管理员完成订单 complete', comp.status === 200 && o.statu
s === 'completed' && Number(o.progress) === 100, `status=$
{o.status}, progress=${o.progress}`);

```

```

    const rev = await api('POST', '/api/orders/submit-review', user.token, { orderId: oShopId, rating: 5, comment: '维修很满意', images: [] });
    o = await getOrder(oShopId);
    log('用户提交评价 submit-review', rev.status === 200 && rev.data?.success, `http=${rev.status}, 评价后status=${o.status}`);
  };
}

```

```

// 7. 取消订单流程
const oCancel = await api('POST', '/api/orders/create', user.token, {
  orderType: 'repair', deviceType: 1, deviceId: 1, problem: '屏幕碎裂', description: '测试取消', serviceType: 'shop', brand: '苹果', model: 'iPhone 13'
});
const oCancelId = oCancel.data?.data?.order_id_numeric;
if (oCancelId) {
  const c = await api('POST', `/api/orders/${oCancelId}/cancel`, user.token);
  const o = await getOrder(oCancelId);
  log('取消订单 cancel (pending->cancelled)', c.status === 200 && o.status === 'cancelled', `status=${o.status}`);
  // 已完成订单不能取消
  const c2 = await api('POST', `/api/orders/${oShopId}/cancel`, user.token);
  log('已完成订单禁止取消(守卫)', c2.status === 400, `http=${c2.status}, msg=${c2.data?.error}`);
}

```

```

// 8. 诊断/扫码 (尽力而为, 不阻断)
const diag = await api('POST', '/api/diagnose/analyze', user.token, { deviceType: '手机', brand: '苹果', symptom: '无法开机', details: '进水' });
log('AI故障自检 /diagnose/analyze', diag.status === 200, `http=${diag.status}, success=${diag.data?.success}, ${JSON.stringify(diag.data?.data?.summary || diag.data?.error || diag.data?.message || "").slice(0,60)}`);
const scanImg = 'D:/maintain/电子维修2.0/uploads/progress/25/1779522114199-38925454.jpg';
if (fs.existsSync(scanImg)) {
  const sfd = new FormData();
  sfd.append('image', new Blob([fs.readFileSync(scanImg)], { type: 'image/jpeg' }), 'verify_scan.jpg');
  const scan = await api('POST', '/api/scan/identify', user.token, sfd);
  log('扫码识别 /scan/identify', scan.status === 200, `http=${scan.status}, ${JSON.stringify(scan.data?.data?.device_type_name || scan.data?.error || scan.data?.message || "").slice(0,60)}`);
}

```

```

// 9. 汇总
console.log(`\n===== 验证汇总 =====`);
const pass = results.filter(r => r.ok).length;

```

```

const fail = results.filter(r => !r.ok);
console.log(`通过: ${pass}/${results.length}`);
if (fail.length) { console.log('失败项:'); fail.forEach(f =>
  console.log(` - ${f.step} | ${f.info}`)); }

// 10. 清理
console.log(`\n----- 清理测试数据 -----`);
const ids = [oShopId, oHomeId, oCancelId].filter(Boolean);
if (ids.length) {
  const ph = ids.map(i => '?').join(',');
  await pool.query(`DELETE FROM order_progress_photos WHERE order_id IN (${ph})`, ids).catch(()=>{});
  await pool.query(`DELETE FROM transaction_income WHERE order_id IN (${ph})`, ids).catch(()=>{});
  await pool.query(`DELETE FROM orders WHERE id IN (${ph})`, ids).catch(()=>{});
}
await pool.query(`DELETE FROM user_addresses WHERE user_id = ?`, [user.id]).catch(()=>{});
await pool.query(`DELETE FROM user_units WHERE user_id = ?`, [user.id]).catch(()=>{});
await pool.query(`DELETE FROM users WHERE openid IN (?, ?)`, [U, A]).catch(()=>{});
console.log(`测试账号/订单/地址/单位已清理(进度照片目录遗留请用 bash rm 清理)`);
await pool.end();
process.exit(fail.length ? 1 : 0);
}
main().catch(e => { console.error('验证脚本异常:', e); process.exit(1); });

```

// ===== backend/verify_photo.js =====

```

const fs = require('fs');
const path = require('path');
const mysql = require('mysql2/promise');
const jwt = require('jsonwebtoken');
require('dotenv').config({ path: path.join(__dirname, './.env') });
require('dotenv').config({ path: path.join(__dirname, '.env'), override: true });
const BASE = 'http://localhost:3001';
async function api(m, u, t, b) {
  const h = {}; if (t) h['Authorization'] = 'Bearer ' + t;
  const o = { method: m, headers: h };
  if (b instanceof FormData) o.body = b; else if (b !== undefined) { h['Content-Type'] = 'application/json';
    o.body = JSON.stringify(b); }
  const r = await fetch(BASE + u, o); let d = {}; try { d = await r.json(); } catch (e) {}
  return { status: r.status, data: d };
}

const pool = mysql.createPool({ host: process.env.DB_HOST, port: parseInt(process.env.DB_PORT)||3306, user:
process.env.DB_USER, password: process.env.DB_PASSWORD, database: process.env.DB_NAME, connectionLimit: 3, c
harset: 'utf8mb4' });
(async () => {
  const openid = 'photo_verify_' + Date.now();
  const c = await pool.getConnection();
  const r = await c.query("INSERT INTO users (openid,nickn
ame,role,status,created_at) VALUES (?,?,?,1,NOW())", [open
id,'照片验证','admin']);
  const uid = r[0].insertId; c.release();

```

```

const token = jwt.sign({ uid: uid, openid, nickname:
'照片验证', role: 'admin' }, process.env.JWT_SECRET, { expires
In: '1h' });
const o = await api('POST', '/api/orders/create', token,
{ orderType:'repair', deviceType:1, deviceId:1, problem:'
测试照片', description:'照片上传验证', serviceType:'shop', brand:'苹果
', model:'iPhone' });
const oid = o.data?.data?.order_id_numeric;
console.log('创建订单:', oid);
const img = 'D:/maintain/电子维修2.0/uploads/progress/25/177
9522114199-38925454.jpg';
const fd = new FormData();
fd.append('orderId', String(oid));
fd.append('description', '复跑验证');
fd.append('images', new Blob([fs.readFileSync(img)], { type: 'image/jpeg' }), 'verify2.jpg');
const up = await api('POST', '/api/progress/photos/upload', token, fd);
const photos = up.data?.data?.photos || [];
console.log('上传返回' http=', up.status, 'success=', up.da
ta?.success, 'photos=', JSON.stringify(photos));
let onDisk = false, localFile = '';
if (photos[0]) {
  localFile = path.join(__dirname, '..', photos[0].replace(/^\//, ''));
  onDisk = fs.existsSync(localFile);
  console.log('落盘校验', onDisk, '->', localFile, fs.exis
tsSync(localFile) ? fs.statSync(localFile).size + ' bytes'
: '');
}
const fb = await api('GET', `/api/progress/feedbacks/${oid}`, token);
console.log('反馈列表' http=', fb.status, '条数=', Array.isAr
ray(fb.data?.data) ? fb.data.data.length : 'N/A');
console.log(up.status===200 && photos.length>0 && onDisk
? '\n结果: 进度照片上传功能 PASS □' : '\n结果: 进度照片上传功能 FAIL □');
await pool.query('DELETE FROM order_progress_photos WHERE order_id=?',[oid]).catch(()=>{});
await pool.query('DELETE FROM orders WHERE id=?',[oid]).catch(()=>{});
await pool.query('DELETE FROM users WHERE openid=?',[openid]).catch(()=>{});
await pool.end();
process.exit(up.status===200 && photos.length>0 && onDisk ? 0 : 1);
})();catch(e => { console.error(e); process.exit(1); });

```

// ===== backend/verify_upload.js =====

```

// 验证图片上传功能 + 共享 uploads 目录
const path = require('path');
const fs = require('fs');
require('dotenv').config({ path: path.join(__dirname, '.env'), override: true });
const jwt = require('jsonwebtoken');
const db = require('./database');

```

```

const BASE = 'http://localhost:3001';

```

```

async function main() {

```

```

// 取测试用户
const users = await db.query("SELECT * FROM users WHERE openid='verify_test_user'");
const user = users[0];
if (!user) { console.log('测试用户不存在'); return; }
const token = jwt.sign({ userId: user.id, openid: user.openid, nickname: user.nickname, role: user.role },
process.env.JWT_SECRET, { expiresIn: '1h' });

// 生成一张最小 PNG
const png = Buffer.from('iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAyAAAAFfcSJAAAADUIEQVR42mP8z8BQDwAEhQGAhKm
MIQAAA
ABJRU5ErkJggg==', 'base64');
const tmp = path.join(__dirname, '_test_upload.png');
fs.writeFileSync(tmp, png);

// 上传单图
const fd = new FormData();
fd.append('file', new Blob([png], { type: 'image/png' }), 'test_upload.png');
const res = await fetch(BASE + '/api/upload/image', {
  method: 'POST',
  headers: { 'Authorization': 'Bearer ' + token },
  body: fd
});
const data = await res.json();
console.log('[上传单图] http=${res.status} url=${JSON.stringify(data.data || data.url || data.path)}');

// 上传多图
const fd2 = new FormData();
fd2.append('files', new Blob([png], { type: 'image/png' }), 'a.png');
fd2.append('files', new Blob([png], { type: 'image/png' }), 'b.png');
const res2 = await fetch(BASE + '/api/upload/images', {
  method: 'POST',
  headers: { 'Authorization': 'Bearer ' + token },
  body: fd2
});
const data2 = await res2.json();
console.log('[上传多图] http=${res2.status} urls=${JSON.stringify(data2.data || data2.urls)}');

// 验证上传目录（容器内 /uploads 与 php、nginx 共享）
const dir = '/uploads';
const exists = fs.existsSync(dir);
const files = exists ? fs.readdirSync(dir).filter(f => f.includes('test_upload') || f.endsWith('.png')) :
[];
console.log('[共享目录核对] /uploads 存在=${exists}, 上传后文件数=${fs.readdirSync(dir).length}');
fs.unlinkSync(tmp);

process.exit(0);
}
main().catch(e => { console.error('异常:', e.message);

```



```
ss.exit(1); });
```

```
// ===== project.config.json =====
```

```
{
  "compileType": "miniprogram",
  "libVersion": "3.15.0",
  "packOptions": {
    "ignore": [
      {
        "type": "file",
        "value": "*.md"
      },
      {
        "type": "file",
        "value": "*.bat"
      },
      {
        "type": "file",
        "value": "*.ps1"
      },
      {
        "type": "file",
        "value": "*.sh"
      },
      {
        "type": "file",
        "value": "*.py"
      },
      {
        "type": "file",
        "value": ".env"
      },
      {
        "type": "file",
        "value": "debug_admin_dashboard.js"
      },
      {
        "type": "file",
        "value": "fix_super_admin_wxml.js"
      },
      {
        "type": "file",
        "value": "model"
      },
      {
        "type": "folder",
        "value": "backend"
      },
      {
        "type": "folder",
```

```
"value": ".claude"
},
{
  "type": "folder",
  "value": ".cloudbase"
},
{
  "type": "folder",
  "value": "uploads"
},
{
  "type": "folder",
  "value": "libs"
},
{
  "type": "folder",
  "value": "assets"
},
{
  "type": "folder",
  "value": "miniprogram"
},
{
  "type": "file",
  "value": "*.html"
},
{
  "type": "file",
  "value": "*.log"
},
{
  "type": "file",
  "value": "e2e_result.txt"
},
{
  "type": "file",
  "value": "维修功能使用流程说明图.svg"
},
{
  "type": "file",
  "value": "维修流程业务逻辑图.mmd"
},
{
  "type": "file",
  "value": "*.mmd"
},
{
  "type": "file",
  "value": "docker-compose.yml"
},
}
```

```

{
  "type": "file",
  "value": ".dockerignore"
},
{
  "type": "file",
  "value": ".env.example"
},
{
  "type": "file",
  "value": ".gitignore"
},
{
  "type": "file",
  "value": "Dockerfile"
},
{
  "type": "file",
  "value": "images/service-avatar-new.png"
},
{
  "type": "file",
  "value": "images/service-avatar-new.svg"
},
{
  "type": "file",
  "value": "images/service-avatar.png"
}
],
"include": []
},
"setting": {
  "coverView": true,
  "es6": true,
  "postcss": true,
  "minified": true,
  "enhance": true,
  "showShadowRootInWxmlPanel": true,
  "packNpmRelationList": [],
  "babelSetting": {
    "ignore": [],
    "disablePlugins": [],
    "outputPath": ""
  },
  "compileWorklet": false,
  "uglifyFileName": false,
  "uploadWithSourceMap": false,
  "packNpmManually": false,
  "minifyWXSS": true,
  "minifyWXML": true,

```

```

    "localPlugins": false,
    "condition": false,
    "swc": false,
    "disableSWC": true,
    "disableUseStrict": false,
    "useCompilerPlugins": false
  },
  "condition": {},
  "editorSetting": {
    "tabIndent": "auto",
    "tabSize": 2
  },
  "appid": "wx005066bc50bb8957",
  "simulatorPluginLibVersion": {}
}
// ===== project.private.config.json =====
{
  "description": "项目私有配置文件。此文件中的内容将覆盖 project.config.json
中的相同字段。项目的改动优先同步到此文件中。详见文档：https://developers.weixin.qq.co
m/miniprogram/dev/devtools/projectconfig.html",
  "projectname": "%E7%94%B5%E5%AD%90%E7%BB%B4%E4%BF%AE2.0",
  "setting": {
    "compileHotReload": true,
    "urlCheck": false,
    "coverView": true,
    "lazyloadPlaceholderEnable": false,
    "skylineRenderEnable": false,
    "preloadBackgroundData": false,
    "autoAudits": false,
    "useApiHook": true,
    "showShadowRootInWxmlPanel": true,
    "useStaticServer": false,
    "useLanDebug": false,
    "showES6CompileOption": false,
    "bigPackageSizeSupport": true,
    "checkInvalidKey": true,
    "ignoreDevUnusedFiles": false
  },
  "libVersion": "3.15.0",
  "condition": {}
}
// ===== sitemap.json =====
{
  "desc": "关于本文件的更多信息，请参考文档 https://developers.weixin.qq.c
om/miniprogram/dev/framework/sitemap.html",
  "rules": [{
    "action": "allow",
    "page": "*"
  }]
}

```